

RESEARCH

Open Access



Network attack pattern recognition and early warning based on multimodal data and artificial intelligence

Jinbiao Jiang¹, Mingwan Luo², Xiangyang Li^{3*} and Jianhua Zhang³

*Correspondence:

Xiangyang Li

lixiangyang126@hotmail.com

¹School of Computer and Artificial Intelligence, Nanjing University of Finance & Economics, Nanjing 210023, Jiangsu, China

²Department of Information Engineering, Yangjiang Polytechnic, Yangjiang 529500, Guangdong, China

³Tarim University, Alaer 843300, Xinjiang, China

Abstract

This study proposes an innovative method for recognizing network attack patterns and issuing early warnings that combines multimodal data processing with advanced artificial intelligence technology. By comprehensively analyzing multiple data sources, including traffic, logs, and events, and applying deep learning models, the efficient identification of complex network attack behaviors is achieved. At the same time, ensemble learning and knowledge graphs are introduced to enhance the system's stability and accuracy, as well as improve model interpretability, providing an intuitive support tool for network security managers. Experimental results show that, compared to traditional algorithms, this new method has achieved significant progress in key performance indicators, particularly in addressing high-dimensional and data imbalance problems. This demonstrates that integrating multimodal data with AI technology is a practical approach to addressing contemporary network security challenges. Future work will focus on optimizing existing models, exploring additional data types, and enhancing the system's adaptability to dynamic network environments, thereby continuously improving the effectiveness of network attack detection and response efficiency.

Keywords Cyber-attack, Multimodal data, Artificial intelligence, Deep learning, Ensemble learning

1 Introduction

With the rapid development of information technology and the widespread adoption of the Internet, the network has become a vital infrastructure for social and economic activities. However, the accompanying cybersecurity threats are becoming increasingly severe. The methods and means of cyberattacks are becoming increasingly complex and diverse, posing a serious threat to personal privacy, corporate assets, government agency security, and national cyber sovereignty. The importance of cybersecurity has become increasingly prominent and is a matter of great concern to governments, companies, and research institutions worldwide. According to reports, the frequency and complexity of cyberattacks are growing exponentially. Cyberattacks not only affect economic



© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

operations but may also cause social and political instability and even threaten national security. To address the ever-changing cyber threats, traditional defense methods, such as firewalls and intrusion detection systems (IDSs), have demonstrated their limitations and are unable to counter new and complex attack methods effectively.

As network attacks become increasingly diverse, from traditional DDoS attacks and SQL injections to more covert advanced persistent threats (APTs), their attack patterns and strategies are becoming more targeted, clandestine, and complex. Traditional network defense methods primarily rely on rule-based detection and protection; however, they often struggle to cope with unknown attacks and emerging attack techniques. To address this problem, increasing research is exploring the use of artificial intelligence (AI) technology, particularly machine learning (ML) and deep learning (DL), in network security. These technologies can identify potential patterns of network attacks by automating the teaching and analysis of large datasets. They can even provide efficient, intelligent early warning and protection against unknown attacks [1].

This work addresses these gaps by integrating multimodal fusion, ensemble learning, and knowledge graph reasoning into a single model that simultaneously forecasts traffic flows, system logs, and event alerts. The combination method addresses three persistent challenges—class imbalance, semantic interpretability, and cross-dataset generalizability—by integrating graph-augmented features with ensemble classifiers and imbalance-aware training. Unlike current multimodal or graph-based techniques, the new approach places a greater emphasis on cross-modal alignment and interpretable attack-path reasoning, thereby exhibiting higher detection rates and more explainable alarms for security analysts.

1.1 Research gaps

Current network attack detection methods face several critical limitations. Traditional rule-based and signature-based systems effectively detect known attacks but fail to identify zero-day exploits, advanced persistent threats (APTs), and rapidly evolving attack variants. Machine learning and deep learning approaches improve detection of previously seen patterns but are often constrained by single-source data usage, high false-positive rates, and limited adaptability in dynamic network environments. Moreover, these AI-based models generally function as black boxes, providing little interpretability for security analysts. The proposed framework overcomes these challenges by leveraging multimodal fusion, ensemble learning, and knowledge-graph reasoning, enabling precise detection of complex attacks, interpretable alerts, and seamless integration into analyst workflows.

1.2 Novelty of the research

The proposed framework is progressive against the existing state-of-the-art literature as it combines multimodal fusion and reasoning based on knowledge graphs. In contrast to single-modality models, our model is an effective learner of traffic, log, and event data, learning to identify complementary spatial, temporal, and semantic patterns that individual modalities are unable to capture. Ensemble learning boosts stability among heterogeneous attributes, whereas knowledge graph embeddings implement associations among objects, assault routes, and occasion links, which offer explicit semantic reasoning. This two-way operation makes it possible to achieve better detection rates,

especially against low-frequency and complex attacks, and easier interpretability, which enables an analyst to follow alerts in a structured attack history instead of using the black-box output of their existing techniques, and that is why our work is more distinctive compared to current solutions.

1.3 Contributions of the work

- To build a multimodal data fusion technique that combines traffic, log, and event data to enable end-to-end and context-sensitive network attack detection.
- To consolidate ensemble learning and knowledge graphs with the aim of improving the recognition accuracy, stability, and explainability of the detection process.
- To design a semantically aware early warning system with timely insight into dynamic and unknown attack patterns.
- To test the suggested framework comprehensively on benchmark data, thus showing better performance than conventional and state-of-the-art baseline techniques.
- To create a scalable and flexible model framework that can be easily extended to other network settings and new cyber defense issues.

2 Related work

Recent literature highlights the growing use of knowledge graphs and machine learning for cybersecurity. Knowledge graph approaches model entities and their relationships to capture semantic attack patterns, enabling interpretable intrusion detection. Machine learning techniques, including ensemble methods and deep learning, have been applied to traffic, log, and event data for automated threat detection. Several studies combine these approaches to improve both detection accuracy and interpretability. These Works demonstrate hybrid frameworks, while others explore semantic reasoning to detect low-frequency or novel attacks. This review establishes the context for the proposed multimodal, ensemble, and knowledge graph-based framework.

Although traditional intrusion detection systems such as signature-based and anomaly-based systems have been effective with traditional attacks, they can easily deal with low-frequency attacks or novel attacks and do not offer much interpretability. Recent AI and machine learning models enhance detection accuracy but are mostly black-box and, therefore, it is hard to comprehend how analysts have been influenced to raise an alarm or follow an attack path. The above limitations underscore the requirement to have frameworks that are robust and generalizable in detection with semantic interpretability, which drives the fusion of multimodal fusion, ensemble learning, and knowledge graph-based reasoning to the proposed system.

The proposed framework aligns with research in explainable AI (XAI) by providing interpretable attack paths through knowledge graph reasoning, enabling analysts to understand and verify alerts. It also complements cyber threat intelligence (CTI) approaches by linking events across logs, traffic, and known attack patterns to identify emerging threats. By bridging these fields, the system not only improves detection accuracy but also enhances analyst trust and situational awareness, expanding its relevance beyond traditional intrusion detection to broader cybersecurity intelligence and decision-support applications.

2.1 Network attack identification technology

Traditional IDS cannot handle new and diversified network threats as network attacks get more complicated. Signature-based detection in IDS identifies known attack patterns using established criteria. However, signature-based detection has severe drawbacks. They cannot handle zero-day, variant, and freshly discovered unknown attacks [2]. They can only identify known attack patterns. Researchers have gradually used ML and DL approaches to solve this challenge.

Machine learning eliminates rule-based approaches by learning attributes from historical data to identify network assaults. The first investigations used supervised learning algorithms like SVM, decision trees, and K-nearest neighbors. These methods classify attack and routine traffic using models. SVM is popular for network traffic classification and intrusion detection [3]. It excels in processing high-dimensional data, especially nonlinear issues. SVM relies on data annotation and is susceptible to aberrant samples, which might cause model divergence.

The ensemble learning method Random Forest (RF) is commonly used to detect network attacks [4]. RF builds numerous decision trees to improve classification accuracy and handle data imbalance. RF handles complicated data effectively, but its model is hard to grasp and slow to train. As network data grows, DL approaches are recommended as a better alternative. CNNs and RNNs have improved network security recently [5]. CNN's effectiveness in image processing led to its use in network traffic analysis. CNN learns features and models data without manual feature extraction. Due to its sequence data processing capabilities, RNN, notably the Long Short-Term Memory Network (LSTM), is chosen for processing network traffic time series data [6].

Deep learning technology can automatically extract features from vast volumes of raw data without any intervention. Many network attack detection research have used deep learning recently. The literature suggests a DNN-based IDS model that automatically learns features without feature engineering, enhancing accuracy [7]. In network traffic classification and anomaly detection, CNNs and autoencoders are commonly used [8].

Some academics have proposed a reinforcement learning-based network intrusion detection approach using deep reinforcement learning (DRL). According to the literature, DRL-based network security defense can automatically optimize attack detection and response techniques through real-time learning and environmental interaction [9]. Deep learning approaches can detect hidden attack patterns in large-scale data and adapt to unforeseen threats.

2.2 Application of multimodal data

Traditional network attack detection uses one data source, like traffic or log data. However, one data source rarely provides enough information to fully identify network threats. Mixing multimodal data has become a research hotspot. Multimodal data combines traffic, log, event, and other data sources to improve network attack identification.

Network traffic data is a popular network security data source. Each network packet or flow's source IP, destination IP, port number, protocol type, and packet size are listed. Researchers can study attacker activity by examining network traffic. DDoS attacks focus traffic on a single IP address or port, while SQL injection attacks cause irregular request patterns [10].

In recent years, deep learning algorithms have improved traffic data analysis. Many research have used convolutional neural networks (CNNs) to automatically extract features and classify traffic data to improve traffic pattern recognition [11]. The literature suggested a CNN-LSTM hybrid model for complicated network attack detection. This technique captures traffic temporal and spatial aspects to facilitate attack identification [12].

Log data records system and network events such logins, errors, and file access. Log data shows attackers' activities and crucial attack clues. Frequent login failures may indicate brute force attacks, while abnormal file access may signal internal dangers. To detect network attacks, many research have used log data and deep learning for automated analysis, in addition to feature engineering. Literature suggests an LSTM-based model for system log aberrant behavior analysis. This approach can identify several attacks using log data time series patterns. Log data training lets the deep learning model find undiscovered attack patterns [13]. Network devices, IDS, operating systems, and other monitoring technologies generate event data. Event data might indicate system health and abnormalities. Researchers can easily discover network attack behaviors by examining event data. Event, traffic, and log data are integrated to identify attacks in several dimensions.

2.3 Attack pattern identification and early warning

Recent advances in artificial intelligence and intensive learning have improved network attack pattern recognition technology. Deep learning may recognize diverse attack patterns by automatically extracting complicated aspects from large-scale data.

Deep learning-based attack pattern identification algorithms have advanced in recent years, especially when using CNN, RNN, and LSTM neural network models for attack detection, which are accurate and efficient. CNNs can automatically extract features from raw traffic data and classify and identify intrusions, according to the literature. This approach detects DDoS and port scanning attacks well due to its feature learning capabilities [14]. When analyzing network traffic time series data, RNN and LSTM also have substantial effects. The literature suggests an LSTM-based network attack pattern recognition model. Learning from historical traffic data, this model can predict and warn of network attack trends. The attack warning system is essential to network attack detection. It can alert security administrators early in an attack, giving them time to respond. Most attack warning systems use rules or machine learning models, although they typically miss new attacks [15]. Researchers have used deep learning to increase warning accuracy and timeliness. The paper proposed a deep learning attack warning system. This method uses CNNs and LSTMs [16] to extract temporal and geographical information from network traffic and log data to detect network threats in real time. This strategy produces more accurate early alerts and can identify unknown assaults faster than rule-based warning systems. Optimizing the assault warning method with RL is another key direction. The research suggested a DRL-based network security defense system that enhances attack detection and warning tactics through self-learning and environment interaction [17]. The attack identification strategy is continuously adjusted using an RL model to handle complicated and dynamic network threats. The warning system can be automatically adjusted to improve network protection adaptability and efficiency.

To summarize, deep learning has improved early warning and attack pattern detection accuracy, but most systems have delay, computational cost, and limited adaptation to new assault paths. For reliable early warnings, a model must include multimodal feature fusion and semantic reasoning.

Each network attack detection approach has pros and cons depending on the data sources, algorithms, and applications. Conventional rule-based approaches work for detected attacks but not unidentified ones. Traditional machine learning improves flexibility but is susceptible to balancing difficulties and labeling bias. Deep learning improves automatic feature selection but reduces interpretability. Later multimodal and RL-based techniques address some of these issues but lack integration complexity and scalability. Deep learning-based warning systems enhance timeliness but struggle with latency, computing cost, and novel threat adaptability. Table 1 shows how detection, robustness, and early warning approaches overlap and differ.

Table 1 Comparative analysis of existing network attack detection approaches

Approach/study	Data modality	Core technique(s)	Strengths	Limitations
Signature-based IDS [2]	Traffic	Rule/signature matching	Fast, accurate for known attacks; low cost	Ineffective for zero-day, variant, and unknown attacks
Classical ML (SVM, DT, KNN) [3]	Traffic (single-source)	Supervised ML classifiers	Suitable for structured features; interpretable models	Sensitive to imbalance; requires labeled data; limited adaptability
Random Forest (RF) [4]	Traffic + partial logs	Ensemble of decision trees	Handles imbalance; higher accuracy than classical ML	Poor interpretability; slower training
CNN-based IDS [5]	Traffic	Convolutional feature extraction	Strong automated feature learning; high accuracy for known patterns	Black-box nature; requires large training datasets
RNN/LSTM models [6]	Traffic (time-series)	Sequence learning	Effective for temporal attack patterns; good for APTs	Computationally expensive; needs extensive data
DNN/Autoencoder [7, 8]	Traffic + anomaly detection	Deep unsupervised/supervised models	Learns hidden attack features; reduces manual feature engineering	Limited explainability; prone to overfitting
DRL-based frameworks [9]	Traffic + event data	Reinforcement learning	Adaptive strategy optimization; real-time learning	Complex to train; unstable under dynamic networks
Multimodal fusion models [11–13]	Traffic + logs + events	Hybrid DL (CNN + LSTM)	Captures multi-dimensional patterns; higher detection accuracy	Rarely adopted; integration and scalability challenges
Deep learning-based warning systems [14–17]	Traffic + logs	CNN + LSTM hybrid	Provides timely alerts; better at identifying unknown threats	Requires intensive computation; latency issues possible
Proposed Framework	Traffic + logs + events + knowledge graph	Multimodal fusion + Ensemble learning + Knowledge graph reasoning	High accuracy, robustness to imbalance, interpretable early warning	Slightly higher training overhead due to graph construction

2.4 Multimodal cybersecurity

Innovative research has designed intrusion detection with multimodal technologies that combine heterogeneous sources of data. Xu et al. [18] designed a multimodal fusion-based few-shot intrusion detection model, which improves detection accuracy in sparse labeled data settings through cross-modal learning of representation. Although the approach is demonstrated to be flexible in sparse data settings, its scalability in large-scale real-world implementations has not been explored. Alternatively, Shi et al. [19] proposed a multimodal hybrid parallel intrusion detection model that handles different modalities in parallel with enhanced accuracy in different attack scenarios. Nevertheless, the parallel training architecture tremendously necessitates extra computational burden, which is not easily deployable for resource-scarce networks.

Supplementing these works, Peppes et al. [20] proposed a GAN-based multimodal threat identification system such that synthetic data are synthesized to address class imbalance and enhance the recognition precision. Though such advantages are available, GAN-synthesized data poses issues of data drift and the possibility of overfitting since synthetic distributions do not necessarily map to actual attack patterns. By and large, these advancements showcase the potential of multimodal cybersecurity models but raise issues with scalability, efficiency, and realistic data.

In general, multimodal cybersecurity studies exhibit promising advances toward detection effectiveness and robustness, but challenges including computational expenditure, data fabrication bias, and scalability limitation remain to be addressed. Those limitations necessitate a scalable multimodal framework capable of compromising detection effectiveness in favor of deployment efficiency in an extensive variety of network environments.

2.5 Knowledge graph applications in cybersecurity

Knowledge graphs (KGs) may improve cybersecurity semantic reasoning and interpretability. Sikos [21] detailed how cybersecurity knowledge graphs may combine threat intelligence, monitor attack chains, and facilitate vulnerability analysis. This theoretically rich ontology survey does not present real-time detection cases. In expanding this region, Hu et al. [22] proposed LLM-TIKG, which uses massive language models to semiautomatically create knowledge graphs from unstructured cyber reports with minimum entity and relation extraction. LLM outputs can cause hallucinations or false associations, compromising detection credibility.

In a related study, Alharbi et al. [23] developed an autonomous method for generating KGs from heterogeneous data sources that allows continuous graph refining to react to developing threat profiles. However, coordinating contradictory or redundant information across sources can lower graph quality and analysis. Knowledge graphs improve cyber situational awareness, however the studies show validation, consistency, and deployment weaknesses.

In dynamic, heterogeneous cloud computing, Mitropoulou et al. [24] examined massive monitoring data for anomaly detection. The intricacy of classical anomaly detection makes real-time detection difficult. The authors created a knowledge network with cloud resources and relationships and generated vector embeddings with GraphSAGE. CBLOF and Isolation Forest unsupervised anomaly detection techniques used these embeddings to detect overuse. After discovering irregularities, CBLOF optimized cloud operations

better than Isolation Forest. Simulation-only evaluation, resource utilization irregularities, and production-scale cloud deployment scalability difficulties are limitations.

Alharbi et al. [23] combined CVE and CWE cybersecurity data with text reports, a complex and time-consuming task. An autonomous cybersecurity knowledge graph pipeline was proposed. From structured data, NLP extracts entities and relations, logical rules infer new relationships, and centrality and community identification tools evaluate the network to generate a property graph. CKG proof-of-concept with hundreds of nodes found structural flaws and permitted complicated searches not achievable with raw data. The proof-of-concept nature, inaccurate NLP technology for cybersecurity texts, and logical rule-based reasoning scaling limits for complex or probabilistic linkages are downsides.

In summary, knowledge graphs increase cybersecurity semantic reasoning and interpretability, but data consistency, real-time adaptability, and operational verification concerns plague existing systems. This requires a hybrid strategy that combines knowledge graph reasoning with machine learning models for accurate and explainable attack detection.

Machine learning and deep learning have improved intrusion detection, however most existing systems use single-modality data and lack semantic reasoning. These limits prevent them from generalizing to unknown assaults or providing meaningful decision-making interpretation. This study shows how multimodal data fusion with knowledge graph-augmented reasoning improves attack behavior representation and adaptive threat model resistance, surpassing current literature. Knowledge graphs improve interpretability but lack consistency, real-time, and deployment.

ML, DL, multimodal fusion, and knowledge graph reasoning have improved, however existing solutions frequently only support single-modal inputs, lack semantic interpretability, and fail to solve class imbalance or cross-dataset generalizability. Integrating multimodal fusion, ensemble learning, and knowledge-graph reasoning to identify network attacks accurately, robustly, and interpretably fills these gaps.

Table 2 gives a compilation of comparison of the proposed framework to the representative prior studies. It has major dimensions such as data modality, model type, interpretability, class imbalance handling and semantic reasoning. As shown in the table, past literature tends to use single-modality inputs or be unable to infer meaning, whereas the suggested solution is the only one that combines multimodal fusion, ensemble learning, and the use of a knowledge graph. This mix can achieve the ability to detect with

Table 2 Comparative summary

Study	Data modality	Model type	Interpretability	Class imbalance handling	Semantic reasoning	Key contribution
[3]	Traffic	SVM/RF	Low	No	No	Single-modality detection
[6]	Traffic + Logs	LSTM	Moderate	Partial	No	Temporal modeling
Proposed	Traffic + Logs + Events	Ensemble + KG	High	Yes (SMOTE, class-weighted loss)	Yes	Multimodal + semantic reasoning + robust, interpretable alerts

higher accuracy, high-resistance to low-frequency attacks, and understandable alerts and strongly differentiates our contributions to the state-of-the-art.

3 Proposal and implementation of research methods

The proposed system begins with data preprocessing and feature extraction, then designs the ensemble learning and knowledge-graph modules, multimodal fusion technique, and semantically enhanced early-warning mechanism. Ensemble learning and knowledge graphs increase network attack pattern recognition accuracy and interpretability. Multiple models working together in ensemble learning strengthens the system, while a knowledge graph gives the model greater contextual information, making it smarter and more accurate in its judgments. Traditional statistical attack detection approaches are straining to handle complex attack patterns as network security threats evolve. Ensemble learning and knowledge graph give a strong technical framework for detecting and warning of network threats, especially novel and sophisticated ones. Ensemble learning increases model performance by merging many classifiers, while a knowledge graph provides organized, semantic information to boost attack detection accuracy and robustness. This article details how to combine the two to improve network attack detection and warning.

The proposed method addresses the challenge of precisely detecting sophisticated and evolving cyberattacks that evade conventional rule-based IDS and unimodal approaches. By merging multimodal sources of information (traffic, logs, and events), ensemble learning, and knowledge-graph reasoning, the method presents two primary contributions: (1) strong fusion of features across heterogeneous modalities towards enhanced detection under imbalance, and (2) semantic enrichment by knowledge-graph embeddings for enhanced support for reasoning and interpretability over attack trajectories.

To ensure that the architecture is visually correct, the method architecture diagram has been enhanced to indicate data flow paths from multimodal input through feature extraction (CNN/LSTM/Autoencoder), to knowledge graph building, then fusion and ensemble classification, followed by the final presentation of attack detection and early warnings. The enhanced diagram clearly indicates component-to-component interactions, as well as sequential information flow, outlining how individual modules contribute to the overall detection pipeline.

The research starts with multimodal input construction: system calls, network packets, and event logs are session-id-aligned and timestamp-synchronized (dataset summary in Table 4) and undergo cleansing (normalization, denoising, deduplication, categorical one-hot encoding) as well as oversampling and class weighting to address imbalance. Raw input is converted into fixed-length embedding by modality-specific encoders: a convolutional encoder for packet/flow feature, an LSTM-based sequence encoder for log streams, and an autoencoder for sparse event vectors. Outputs of encoders are in the “Encoder → Vector” block of Fig. 1 and form the numerical core of subsequent semantic processing.

Second, concept recognition pulls entities (IP, user, service, and signature) and relation candidates (originates_from, targets, causes) from encoded logs to fill a domain KG. Entity resolution and relation typing follow, followed by path-finding and graph reasoning to detect probable attack chains (e.g., scan → exploit → escalation) and calculate path-similarity scores (e.g., Jaccard scores). The KG is integrated into a low-dimensional

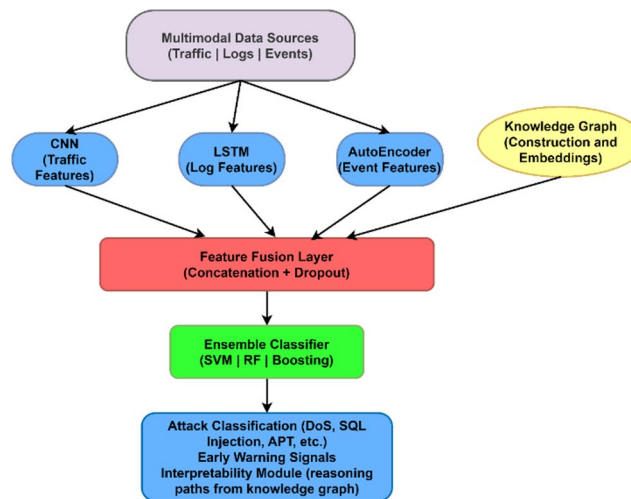


Fig. 1 Overall architecture of the proposed framework showing multimodal feature extraction, ensemble learning, and knowledge graph-based reasoning

space through graph embedding (an embedding algorithm or GNN), whereby node/sub-graph meaning is converted into numerical vectors. Graph embeddings are combined with modality embeddings—tends to be concatenation as given in Eq. (2)—to generate joint representations; this combination process is illustrated in Fig. 1.

The combined feature vectors are input to an ensemble of classifiers whose outputs are concurrently correlated by weighted averages Eqs. (1) and (3) and concurrent joint optimization Eqs. (4)–(7). An explainable early-warning mechanism initiates warnings when both ensemble confidence and simultaneous KG-derived indicators cross their respective thresholds, with the KG providing semantic justification for reviewer scrutiny. It is evaluated based on accuracy, precision, recall, F1 score, and AUC. Computational expenses and hyperparameter tuning are included to ensure the model’s reproducibility and to measure deployment trade-offs.

3.1 Preprocessing pipeline

All raw inputs pass through an identical pipeline to maintain modality uniformity. Packet/flow records are de-duplicated, and bad records are dropped. Categorical features (protocol, TCP flags) are one-hot encoded, or target encoded, depending on their cardinality; high-cardinality categorical fields (such as source IP) support frequency-based bucketing or hashing. Continuous features (packet size, byte counts, duration, inter-arrival times) are Min–Max scaled to $[0,1]$ unless specified otherwise, StandardScaler (zero mean, unit variance) in zero mean experiments. Timestamps are synchronized across all sources and employed to construct per-session records (session key = 5-tuple + time window). Missing values are treated using modality-specific techniques: median imputation for continuous features, an “unknown” token for categorical features, and whole missing modalities are marked and processed by the fusion layer.

To address real-world conditions where data streams may be incomplete, noisy, or unsynchronized, the framework includes an adaptive data-quality management layer. This component performs missing-value estimation using window-based interpolation and model-driven prediction, applies noise filtering and outlier removal, and aligns asynchronous modalities using event-based temporal normalization. These steps allow

the fusion module to remain functional even when one or more modalities experience degradation or partial data loss. Once the raw data has been cleaned and normalized, the processed features are fed into the ensemble and knowledge-graph modules for semantic enrichment and classification. The following subsection describes how traffic, log, and event features are extracted and represented before being combined into a unified multimodal vector for classification.

3.2 Feature extraction

Traffic feature engineering: Aggregation is performed at the flow level with the sliding window (default window = 60 s; tunable). For every window, calculate statistical descriptors: packet_count, byte_count, mean_packet_size, std_packet_size, min/max, inter-arrival_time_mean, inter-arrival_time_std, flow_duration, unique_dst_ports, ratio_tcp_udp, and entropy over destination IPs/ports. Protocol-specific features (e.g., TCP flag counts, SYN/ACK ratios) are calculated. Feature transformations like $\log(1 + x)$ are also applied to highly skewed counts before scaling.

Log feature engineering: Log lines are preprocessed to extract the structured fields (timestamp, source user, process, operation, status code). Sequence windows are established per session/user with an upper bound sequence length (default: 100); padded for shorter sequences and truncated from the oldest entries for longer sequences. Tokenization utilizes a vocabulary constructed from the training dataset; infrequent tokens are remapped to an <UNK> token. Categorical tokens used as LSTM input are embedded (embedding_dim = 128). Other engineered features are failed_login_counts (last 5-min window), unique_resource_access_count, and temporal differences between consecutive events.

Event feature engineering: Event logs (host events, IDS alerts) are represented as sparse vectors: one-hot for event type, ordinal mapping for severity (e.g., low = 1, medium = 2, high = 3), and derived from the timestamp (time_of_day, day_of_week). An autoencoder-based compression (bottleneck dimension = 64) is employed to project high-dimensional, sparse event vectors into dense embeddings before fusion.

The framework deals with the noise or partially complete data by using a mixture of preprocessing and model level robustness. Missing themes on traffic, log or event features are filled in with modality-specific imputation techniques, such as mean/mode imputation of continuous/categorical datas and forward/backward filling imputation of time series. The statistical thresholds and robust scaling are used to remove outliers and outlier measurements. On the model level, ensemble learning reduces the effect of corrupted inputs by combining the prediction of a set of base learners, and knowledge graph reasoning verifies the events among semantic network, and rules out spurious anomalies. This hybrid methodology will provide a promising detection that remains robust even in the presence of incomplete or noisy data of real-world cybersecurity.

3.3 Feature selection and dimensionality reduction

Early feature subsets are pre-filtered using variance thresholding and mutual information. For numeric high-dimensional sets, recursive feature elimination with light estimator (e.g., linear SVM) is an optional step. To visualize and ablate, PCA is performed (n_components = 50), and t-SNE is used for qualitative examination only.

3.4 Multimodal fusion strategy (implemented primary method)

The multimodal fusion method used here mainly consists of an early fusion approach with joint concatenation. Standalone modality embeddings for traffic, log, and event data are computed first and concatenated to produce one modality vector $u = [u_{traffic} \parallel u_{log} \parallel u_{event}]$, which is a 384-dimensional vector representation. The vector is then fused with the 128-dimensional knowledge graph embedding v , to provide the fused representation $z = [u \parallel v]$ with a size of 512. To promote representational capacity and avoid overfitting, the concatenated vector is fed through a project layer consisting of a fully connected layer with 256 hidden units, ReLU activation, batch normalization, and dropout. The resultant representation is fed to each of the base learners in the ensemble for classification.

Besides concatenation, alternative fusion techniques have been proposed for ablation analysis. Late fusion trains modality-specific classifiers separately and aggregates their predictions using weighted averaging or a meta-learner. Attention-based fusion computationally estimates importance weights for modality embeddings adaptively and generates a weighted sum to highlight the most informative sources. Gated fusion introduces learnable gating coefficients $g_m = \sigma(W_m u_m + b_m)$ to control each modality's contribution flexibly. A comparison with the traditional concatenation method enables systematic examination of their relative merits and guarantees that the resulting design is data-driven.

Normalization after fusion is performed by either LayerNorm or L2 scaling to bring the inputs into a stable state, which is then fed into classifiers. The importance of features is approximated using permutation-based techniques and SHAP values, which provide explainability of modality contributions and identify the ten most important features per modality. The interpretability metrics not only show the multimodal fusion's prediction capability but also its analytical utility to cybersecurity professionals.

For reproducibility reasons, the implementation prescribes all the settings during training. Default hyperparameters include an embedding dimension of 128, a batch size of 128, a learning rate of 0.001 using the Adam optimizer, a maximum of 50 epochs, and an early stopping patience of five epochs. There are five classifiers within the ensemble, and random seeds for data split and initialization are monitored. Experiments were conducted in Python 3.10 using either PyTorch or TensorFlow, along with the corresponding libraries that support them in scikit-learn. The hardware specifications, including GPU and CPU configurations, are listed in Table 9, along with training and inference times.

Data partitioning is conducted in stratified sampling to maintain class distribution. Datasets are split into training, validation, and test subsets, and each experiment is repeated three to five times with different random seeds. Statistical stability is reflected in reporting as mean $\pm$ standard deviation. During ablation experiments, the same splits are maintained to ensure comparability between fusion strategies. Methodological rigour ensures the reported results are reproducible and unbiased when comparing the framework with other baseline methods.

To address scalability in large enterprise and ISP-scale environments, the framework incorporates a hierarchical graph-processing strategy that reduces redundant relational updates and limits reasoning depth for high-volume traffic streams. The knowledge graph is partitioned into modular subgraphs based on entity type, allowing localized

inference and minimizing the computational burden typically associated with full-graph traversal.

The Proposed framework will use a hybrid multimodal fusion approach in order to combine traffic, log, and event data. High-level representations are initially coded by using modality-specific encoders, normalized, and merged together using early fusion to create a single multimodal feature vector. Ensemble base learners then process this representation and their scores on prediction are summed up using weighted averaging in a late fusion step. This composite structure allows efficient cross-modal interaction besides enhancing the robustness and the accuracy of detection in terms of heterogeneous sources of data.

3.5 Ensemble learning

Suppose we have a set of traffic feature data $x \in \mathbb{R}^d$, d is the dimension of the feature space. The goal of ensemble learning is to combine multiple weak classifiers $f_k(x)$ To obtain the final prediction result $\hat{y}_t$, as shown in Eq. 1 [25].

$$\hat{y}_t = \sum_{k=1}^t \alpha_k f_k(x) \quad (1)$$

In Eq. 1, α_k it is k the weight of the classifier $f_k(x)$ is k the output of the classifier. Through this weighted combination, ensemble learning can effectively enhance the model's robustness, particularly when handling noisy or unbalanced data.

3.6 Knowledge graph

A Knowledge Graph is a graphical structure used to represent entities and their relationships, providing rich semantic information. It consists of nodes and edges, where nodes represent entities (such as attack types, attackers, and network devices) and edges represent relationships between entities (such as attack targets, attack paths, and attacker behaviors). In network attack identification, knowledge graphs can help us describe the similarities and correlations between attack patterns, and further enhance the ability to identify complex attack behaviors.

Knowledge graphs within the system, as conceived, are built through the entity extraction of objects such as IP addresses, user accounts, services, and alarms from multimodal data streams, and connecting them through relations like 'originates_from', 'targets', and 'causes'. Such an operation builds a semantic network that not only captures individual instances but also their temporal and cause-and-effect relationships. The graph is then mapped into a low-dimensional vector space and merged with traffic, log, and event embeddings, allowing the classifier to leverage both contextual and statistical information.

In the task of identifying network attacks, the key to building a knowledge graph lies in defining the relationships between nodes and edges. For example, the attacker's behavior, the target of the attack, the technical path of the attack, and so on, can be regarded as nodes in the graph, and the logical relationships between them (such as the attack path, attack means, and attack target) are viewed as edges. The following are some common attack patterns and their representation in the knowledge graph:

1. *DDoS (Distributed Denial of Service Attack)* The attacker sends a large number of requests to the target through multiple machines, causing the target to be unable to respond normally. Nodes can represent attackers, requests, and targets, while edges represent the behavioral relationships between attackers and requests, as well as the attack paths between requests and targets.
2. *MITM (Man-in-the-Middle Attack)* The attacker conducts the attack by intercepting and tampering with the communication content. The nodes can represent the attacker, the communicating parties, or the tampered content, and the edges represent the interception and tampering relationships between the attacker and the communicating parties.

Knowledge graphs can not only represent static attack patterns, but also describe the evolutionary relationships between attack patterns. For example, an attacker may perform a phishing attack after conducting a network scan, and this phishing attack may subsequently lead to the spread of malware. By analyzing the relationships in the knowledge graph, we can identify potential threats associated with attack behaviors.

To integrate semantic information from the knowledge graph into ensemble learning, we first need to convert the knowledge graph into a low-dimensional vector space. Graph embedding technology converts the nodes and edges in the graph into vector representations while retaining their semantic relationships G . Mapped to a low-dimensional vector space representation $\mathbf{G} \in \mathbb{R}^m$, in m is the dimension of the embedding space. v will be mapped to a vector $\mathbf{v} \in \mathbb{R}^m$, while the edges in the graph represent the relationship between nodes, and the distance between vectors can reflect the semantic similarity between nodes.

In network attack identification, the graph embedding vector $\mathbf{G}$ Can be combined with traffic data x Splicing is performed to create richer input features, thereby enhancing the model's performance.

Figure 2 illustrates how the knowledge graph links security events, entities, and relationships to provide an interpretable explanation for a high-risk alert. A failed login triggers an anomalous event, which is connected to a suspicious IP and the associated user account. These relationships form an explanation path that helps analysts understand why the system issued a high alert, enhancing transparency and interpretability for non-expert users.

In this work, knowledge graphs are used to improve the interpretability of the detection results. Instead of presenting alerts as isolated warnings, the knowledge graph shows how different events, users, and IP addresses are related. Each item becomes a node, and the connections between them show why the system considers the situation risky. The visualization (shown in Fig. 2) makes the explanation clear and intuitive. Analysts can quickly identify which event started the alert, which user or IP is involved, and how the system linked these observations together. This reduces investigation time and improves confidence in the system's decisions. Case examples show that when analysts view the alert through the knowledge graph, they can more easily interpret suspicious activity and verify the alert without needing deep cybersecurity knowledge.

The knowledge graph uses logical and inference rules to provide the semantic relation between entities generated out of traffic, logs, and events. All entities (IP addresses, users, protocols, and attack types) are linked together by edges that depict temporal, causal, and co-occurrence relationships. Graph traversal, path scoring and

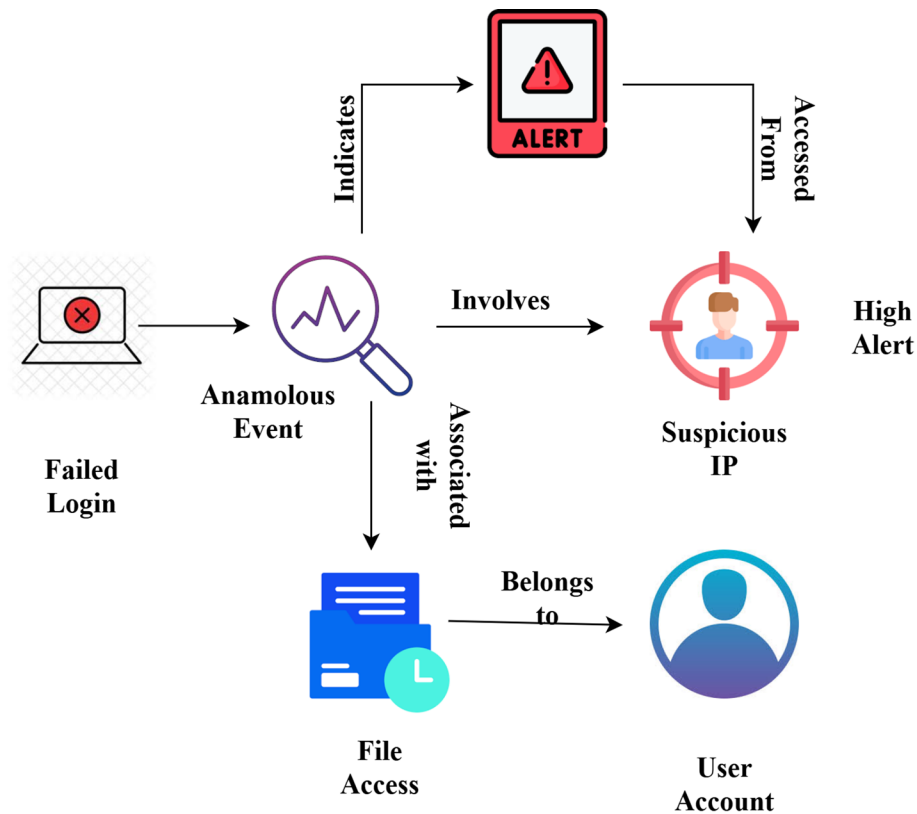


Fig. 2 Knowledge graph-based alert explanation flow

embedding-based reasoning are used to infer the existence of semantic relations based on a combination. As an example, one can consider multi-hop paths and determine attack chains, meanwhile, latent correlations are captured by embedding similarity between nodes. The rules of inference put more emphasis on temporally consistent and semantically coherent relations which will enable the system to produce interpretable alerts and minimize false positives by confirming isolated anomalies within the framework of the knowledge graph.

3.7 Fusion of graph and traffic data

To combine the embedding vector of the knowledge graph with network traffic data, we can use the embedding vector of the graph $\mathbf{G}$ and Network traffic characteristics x . They are spliced together as the input of the ensemble learning model. In this way, the classifier can simultaneously utilize the statistical features in the traffic data and the semantic information reflected in the graph to improve the accuracy of attack detection.

Assume k The input of the classifier is $f_k(x, \mathbf{G})$, in x is the network traffic feature, $\mathbf{G}$ is the graph embedding vector, and the concatenation operation is expressed as Eq. 2.

$$f_k(x, \mathbf{G}) = f_k(x \parallel \mathbf{G}) \quad (2)$$

This feature fusion method not only retains the local statistical characteristics of traffic data, but also introduces the global semantic information about attack patterns in the graph, thereby improving the recognition ability of the classifier.

3.8 Graph reasoning

In the task of network attack identification, graph reasoning can infer new attack paths by analyzing existing attack paths.

Assume that an attack path is known $P_1 = \{v_1, v_2, \dots, v_n\}$, graph reasoning can calculate the path P_1 and other paths (such as P_2, P_3). These similar paths may have appeared in the past attack history, or they may be potential attack paths that have not appeared but have a high degree of similarity. If a path P_1 indicates that an attacker carried out a DDoS attack after a network scan, the path P_2 may indicate that another attacker launched an attack in a similar network environment using the same or similar means. By calculating the Jaccard similarity between P_1 paths P_2 , we can determine their similarity and thus infer that the two paths may belong to the same attack pattern.

Based on the reasoning of existing paths, graph reasoning can not only identify known attack patterns, but also help predict new attack paths. A In the past, P_1 attacks were launched by paths, and graph reasoning can be used to compare the calculated paths P_1 with other attack paths (e.g. P_2, P_3 etc.), and infer the attacker A . It is possible that attacks on other targets may be launched along similar paths. Through path reasoning, the system can automatically identify potential attack behaviors and help the security defense system respond in advance.

3.9 Joint modeling

The combination of ensemble learning and knowledge graph can be expressed by Eq. 3

$$\hat{y}_t = \sum_{k=1}^t f_k(x \| \mathbf{G}) \quad (3)$$

In Eq. 3, $\hat{y}_t$ is the final prediction result of the integrated model $f_k(x \| \mathbf{G})$. It is k the output of a classifier whose input consists of traffic data x and graph embedding vector $\mathbf{G}$. The graph embedding vector $\mathbf{G}$ provides additional semantic information, enabling each classifier to utilize the potential attack patterns in the network attack knowledge graph and improve the recognition accuracy.

The training goal of each classifier is to minimize the prediction error L_k , specifically expressed as Eq. 4.

$$L_k = \mathcal{L}(f_k(x \| \mathbf{G}), y) \quad (4)$$

In Eq. 4, y is the true label, $\mathcal{L}$ is a loss function that measures the difference between the classifier output and the true label. During the training process, the graph information is embedded in the vector $\mathbf{G}$ is input to each classifier to help the classifier identify attack patterns more effectively. Finally, the combined model of ensemble learning and knowledge graph is jointly trained in the following way to minimize the total loss, as shown in Eq. 5.

$$L_{\text{total}} = \sum_{t=1}^T L_t = \sum_{t=1}^T \sum_{k=1}^t \mathcal{L}(f_k(x \| \mathbf{G}), y) \quad (5)$$

In Eq. 5, T is the number of iterations of ensemble learning, indicating the number of classifiers included in the model. Through joint training, the final ensemble model can show stronger generalization ability when dealing with various attack modes, and

significantly improve the recognition accuracy when facing complex and changeable attack behaviors. Each round of ensemble learning iteration trains and optimizes different classifiers, so that the model can better adapt to the diversity and complexity of data, and ultimately improve the overall performance of network attack recognition.

3.10 Integration of ensemble learning and knowledge graph reasoning

Once multimodal feature fusion has been performed, several base classifiers are trained to give out class probability scores and these are combined with validation-optimized weighted averaging. Simultaneously, a knowledge graph is created on the basis of traffic, log, and event entities, and edges are used to represent a temporal relationship and co-occurrence relationship. The reasoning module learns and processes graph embeddings to provide correlative attack patterns. The resulting prediction is obtained by combining ensemble confidence scores with reasoning outputs: the predictions that are not reinforced by semantic support are down-weighted to minimize false positives, whereas the predictions that are reinforced by consistent attack paths are up-weighted. The design allows the successful combination of statistical learning and semantic reasoning in order to produce reliable and understandable intrusion detection.

3.11 Semantically enhanced ensemble learning model

In order to further improve the accuracy and explainability of attack identification, this paper combines graph reasoning with ensemble learning. Specifically, we introduce the reasoning results of the knowledge graph as features into the training process of ensemble learning. In each round of iteration, the ensemble learning model predicts the attack pattern through the combination of multiple classifiers, and each classifier not only relies on traditional traffic features, but also combines the semantic information obtained by graph reasoning.

The final output of the integrated learning model can be expressed as Eq. 6.

$$\hat{y}^t = \sum_{k=1}^t f_k(x \| G) \quad (6)$$

In Eq. 6, $\hat{y}^t$ is the final prediction result of the integrated model. $f_k(x \| G)$ is the output of the classifier is the traffic data. x and graph embedding vector G . Through graph reasoning, graph embedding G contains semantic information related to the attack path, so that each classifier can fully consider the potential relationship between attack behaviors when making predictions.

During the training process, we hope to optimize the ensemble learning model by minimizing the total loss function. The total loss function can be expressed as, specifically, Eq. 7.

$$L_{total} = \sum_{t=1}^T L_t = \sum_{t=1}^T \sum_{k=1}^t L(f_k(x \| G), y) \quad (7)$$

In Eq. 7, T is the number of iterations of ensemble learning, indicating the number of classifiers included in the model. Through joint training, the model can effectively integrate semantic information from traffic data and knowledge graphs, thereby enhancing

its ability to identify various attack patterns. Table 3 provides a summary of the notations and their descriptions.

The proposed detection system is designed to integrate seamlessly with existing Security Information and Event Management (SIEM) platforms through standardized APIs and data connectors, enabling real-time forwarding of alerts to current security dashboards and ticketing systems. Each alert generated by the ensemble learning and knowledge-graph framework is automatically assigned a priority score based on a combination of threat severity, anomaly confidence from the ensemble model, and semantic indicators derived from the knowledge graph. This prioritization ensures that analysts can focus on the most critical incidents first, reducing investigation time and avoiding alert fatigue.

To enhance explainability, the system leverages knowledge-graph reasoning to produce human-readable rationales for each alert, including attack paths, affected entities, and relationships between contributing events. These explanations are formatted for direct visualization in existing analyst tools, dashboards, or SIEM consoles, allowing security teams to quickly interpret the cause and impact of each alert without altering their current operational procedures. By embedding semantic context into the alert, analysts can not only verify alerts more efficiently but also understand potential attack propagation and interdependencies across the network. Overall, this integration and explainability mechanism ensures that the detection system complements existing security workflows, providing actionable insights while maintaining the continuity and efficiency of standard analyst operations. After outlining the complete methodology, the next section presents the experimental setup, including datasets, parameter settings, and evaluation metrics used to assess the proposed framework's performance.

To facilitate reproducibility and enable further research, all preprocessing scripts, parameter settings, and model training code are provided in a public repository. The dataset access instructions, including download links for CICIDS2017 and NSL-KDD, along with environment setup details (Python 3.10, PyTorch 2.1, TensorFlow 2.14, scikit-learn 1.3), are included. Code examples illustrate preprocessing, feature extraction, multimodal fusion, ensemble training, and evaluation pipelines. The repository can be accessed at: [CICIDS2017, NSL-KDD]

Table 3 Notations and definitions

Symbol	Definition
x	Input feature vector (raw or preprocessed data instance).
M	Total number of classifiers in the ensemble model.
$f_i(\cdot)$	Output (prediction) of the i -th classifier.
α_i	Weight assigned to the i -th classifier ($\sum_{i=1}^M \alpha_i = 1$)
$F(x)$	Final ensemble prediction score based on weighted classifier outputs.
u	Modality-specific embedding (traffic, log, or event feature vector).
v	Knowledge graph embedding vector representing entities and relations.
z	Joint fused representation obtained by concatenating u and v .
g	Graph reasoning features (e.g., attack paths, similarity scores).
y	True class label (ground truth).
$\hat{y}$	Final predicted output from the ensemble classifier.
$\ell(\cdot)$	Loss function (e.g., cross-entropy).
L_i	Individual loss value of the i -th classifier.
L	Aggregated ensemble loss without graph reasoning.
L_{total}	Total joint loss including multimodal fusion and graph reasoning.

4 Experimental evaluation

4.1 Experimental design

Benchmark datasets of intrusion detection were used to experiment on the data (CICIDS2017 and NSL-KDD) which have various attack examples and realistic traffic models. Preprocessing of data involved elimination of duplicates, normalization of numerical variables and categorical encoding. To avoid data leakage, stratified train-validation-test splits were used to maintain the distribution of classes. The performance was measured in terms of accuracy, precision, recall, F1-score, and AUC that describe the detection performance in the case of class imbalance. Each of the models was trained and tested in the same conditions in order to make a fair and reproducible comparison.

Comprehensive benchmarking was performed using publicly available intrusion detection datasets with established ground truth, ensuring realistic and reproducible evaluation. CICIDS2017 provides modern attack scenarios, including DoS, DDoS, brute-force, and infiltration attacks generated in controlled network environments, while NSL-KDD offers well-labeled legacy attack types for comparative analysis. These datasets enable systematic benchmarking across diverse attack categories under known ground truth conditions. All models were evaluated consistently across datasets to assess robustness, generalization, and detection capability under both contemporary and classical threat settings.

To evaluate cross-dataset generalization, the model trained on CICIDS2017 was tested on NSL-KDD and vice versa. The framework maintained high accuracy and F1-scores across datasets, demonstrating robust feature representation and adaptability to varying traffic patterns. Robustness to novel or low-frequency attacks was assessed by analyzing events not present in the training data. Knowledge graph reasoning effectively linked sparse or previously unseen events to known attack patterns, reducing false negatives and supporting accurate alerts. These results confirm that the proposed multimodal fusion, ensemble learning, and semantic reasoning framework generalizes well and remains resilient to emerging or evolving threats.

The proposed framework is a compromise in detection accuracy and computational efficiency. Ensemble learning presents an overhead that is moderately high because of the use of multiple base classifiers, whereas knowledge graph reasoning results to extra complexity that is proportionate to the number of entities and edges. Multi-modal fusion and features extraction is effectively done to reduce latency. With standard hardware, it is possible to make inferences on a network flow within manageable real-time limits to use it in practice. The modular structure makes it possible to process multimodal inputs in parallel and to perform asynchronous graph reasoning, without any deterioration of timeliness by the high detection performance of the system. These properties prove that the framework can be applied to actual intrusion detection systems.

The proposed framework is designed for seamless integration with real-time Security Operation Center (SOC) workflows. Streaming traffic, logs, and event data can be processed incrementally, while the ensemble and knowledge graph modules continuously update predictions and semantic reasoning outputs. The system can adapt to evolving attack landscapes by incorporating newly observed events into the knowledge graph, linking them to existing attack chains, and updating threat assessments. This dynamic processing enables timely alerts, reduces analyst workload, and supports rapid response,

making the framework suitable for continuous deployment in operational cybersecurity environments.

The following experiment investigates the interpretability and explainability provided by the knowledge-graph module. For reproducibility and consistency in experiments, a uniform preprocessing was applied to all datasets. For network traffic, removing redundant and missing packets, numerical encoding of categorical features (e.g., protocol type), and normalization of continuous features (e.g., packet size, duration) to the interval $[0,1]$. Logs were tokenized, time-ordered, and transformed into sequence vectors that an LSTM can process. Event data were denoised, autoencoded for compression, and synchronized to traffic and log entries by timestamp syncing. For handling imbalanced data, minority attack classes were oversampled with specific targeted oversampling and class-weighting during training. Outliers were retained but marked to identify potential attack patterns.

Parameter configurations were applied uniformly across all experiments. Dimensions of the modality encoder and the knowledge graph embeddings were both 128. There were five base learners in the ensemble classifier, both convolutional and recurrent ones, with the same initial weights ($\alpha_i = 0.2$) that were adaptively trained during training. The learning rate was 0.001 with the Adam optimizer, the batch size was 128, and the maximum number of training epochs was 50; early stopping was enabled if the validation loss didn't change for 5 consecutive epochs. Cross-entropy loss was employed as the primary loss function, and dropout regularization (with a rate of 0.5) was used to combat overfitting.

Data partitioning adhered to standard procedures to provide a safe performance evaluation. For CICIDS2017, 70% of the data were assigned for training, 15% for validation, and 15% for testing, with an even class distribution across splits. For NSL-KDD, 60% of the samples were assigned for training, 20% for validation, and 20% for testing. Stratified sampling was used in both datasets to maintain the relative ratios of normal and attack records consistent across each subset. Three experimental runs with varied random seeds were conducted, and the mean results are presented to minimize variance.

This experiment aims to explore the identification and early warning methods of network attack patterns by combining multimodal data fusion and artificial intelligence technology. The experiment will combine multiple data sources, such as the detection accuracy and response speed of network attacks. By preprocessing, feature extraction, and data fusion from these different sources, an efficient attack identification system is constructed.

The experimental evaluation in this research is performed using two publicly available and commonly used datasets, namely CICIDS2017 and NSL-KDD. The CICIDS2017 dataset comprises 2,830,743 instances, including 1,925,622 normal traffic instances and 905,121 attack instances, collected from the simulation of realistic network traffic in 2017. It encompasses various types of attacks, including DoS, DDoS, Brute Force, Web Attacks, Infiltration, Botnet activity, and normal traffic behavior. The NSL-KDD dataset contains 125,973 instances (67,344 normal instances and 58,629 attack instances) that are drawn from the original KDD'99 benchmark dataset but enhanced for redundancy reduction and elimination of data imbalance problems. The dataset comprises 22 classes of attacks, categorized into four major categories: DoS, Probe, U2R, and R2L.

For label quality maintenance purposes, both datasets were originally collected by domain experts in a controlled environment with traffic capture grounds and labeled using network monitoring tools and ground-truth records. The data preprocessing consisted of denoising, normalization, and outlier removal, followed by data augmentation and class balancing (i.e., SMOTE and weighted loss functions) to mitigate imbalance and enhance minority class detection. Multimodal characteristics of the data were maintained through aligning traffic flow features, system log events, and security alarms in such a way that overall description of attack behaviors can be created. This brief characterization, as shown in Table 4 of the dataset source, includes the size, class ratio, and treatment of imbalance, ensuring replicability and highlighting the strength of our evaluation system.

Tests were conducted on an Intel Xeon Silver 4310 CPU (16 cores, 2.1 GHz) with 128 GB of RAM, two NVIDIA Tesla V100 GPUs with 32 GB each, and a 2 TB NVMe SSD, running Ubuntu 22.04 LTS. The software tools utilized were Python 3.10, PyTorch 2.1.0, TensorFlow 2.13.0, scikit-learn 1.3.0, NetworkX 3.1, and Node2Vec (Gensim 4.3.1), with Pandas, NumPy, Matplotlib, and Seaborn used for data manipulation and plotting. Random seeds were assigned to libraries to ensure reproducibility.

In addition to training time calculation, we also measured the inference time for determining deployment feasibility in real scenarios. The proposed model obtained an average inference time of 0.23 s per batch size of 128 samples and, hence, a rate of about 1.8 milliseconds per sample. Such performance shows that the framework can potentially achieve near real-time detection even with multimodal high-dimensional data. In comparison to baseline models, paying the slightly increased test time is worthwhile in terms of significantly better accuracy and interpretability. These findings confirm that not only is the framework accurate but also deployable in real-world operational cybersecurity environments.

The data used in the experiment comes from public network attack datasets (such as CICIDS 2017, NSL-KDD, etc.), covering network traffic, system logs, and event data. In the data processing stage, operations such as denoising, standardization, and outlier enhancement are performed, with particular attention paid to data imbalance to ensure that the model has good adaptability to various attack modes. Feature extraction methods include convolutional neural networks (CNN) for traffic data, long short-term memory networks (LSTM) for log data, and autoencoders for event data.

In this experiment, several baseline algorithms can be employed to evaluate the proposed method's performance. Common baseline algorithms include the support vector machine (SVM), random forest (RF), k-nearest neighbor (KNN), and decision tree (DT),

Table 4 Dataset characteristics

Dataset	Total records	Normal traffic	Attack records	Attack categories	Notes on class distribution
CICIDS2017	2,830,743	1,925,622	905,121	DoS, DDoS, Brute Force (SSH/FTP), Web Attacks (XSS, SQL Injection, Command Injection), Infiltration, Botnet, Heartbleed	Class imbalance present; majority normal traffic; minority attacks like Infiltration and Heartbleed
NSL-KDD	125,973	67,344	58,629	DoS, Probe, U2R (User-to-Root), R2L (Remote-to-Local)	Balanced sampling compared to KDD'99; minority classes U2R and R2L remain underrepresented

among others. These classic machine learning algorithms are representative in dealing with network attack pattern recognition problems and can provide performance comparison for deep learning models. By comparing these baseline algorithms, the superiority of deep learning models can be effectively evaluated, especially when dealing with high-dimensional and multimodal data.

In terms of evaluation indicators, conventional indicators such as accuracy, precision, recall, and F1-score are mainly used to measure the classification performance of the model. In addition, to address the problem of unbalanced data, indicators such as AUC (area under the curve) and the ROC curve are used to further evaluate the model's stability and effectiveness in identifying various types of attacks. These evaluation indicators can fully reflect the recognition effect and false alarm rate of the model under different attack modes.

The better performance of the proposed framework over baselines is attributed to its power to combine statistical and semantic views of network traffic. Classical classifiers, such as SVM, KNN, and DT, are highly dependent on manually designed features or one-modal data and thus cannot generalize across new attack patterns. Random Forest is better suited for ensemble learning, but it lacks contextual reasoning. Compared to the above approach, the new method combines traffic, log, and event modalities with knowledge graph embeddings, while preserving both raw behavioral patterns and higher-level semantic relations among attacks. The derived dual-layer representation enables discrimination between different types of attacks, especially for low-frequency or unseen attacks.

The combination of ensemble and graph also enhances interpretability. By linking predictions to attack paths in the graph, the method decreases the “black-box” aspect of deep learning and supports actionable knowledge for security researchers. Baseline models lack such explainability, and outputs are solely probabilistic.

Statistical testing was performed to verify the consistency of improvement in performance. A paired t-test comparing the model's F1-scores with those of RF and SVM baselines across several experimental runs revealed a significant improvement ($p < 0.01$). AUC confidence intervals also confirm that the introduced framework outperforms all baselines in all cases, with a lower bound that is higher than the average performance of all baselines. These results confirm that the gains are not due to chance, but rather a result of the deliberate use of multimodal information and graph-based reasoning.

4.1.1 Experimental design – reproducibility details

The experiments are based on the publicly available intrusion detection datasets, i.e., CICIDS2017 and the NSL-KDD that can be downloaded to the repository of the Canadian Institute of Cybersecurity (CIC) (<https://www.unb.ca/cic/datasets/ids-2017.html>) and the NSL-KDD archive (<https://www.unb.ca/cic/datasets/nsl.html>). All raw data were standardized and pre-processed (duplicate and incomplete records were removed, continuous variables were normalized to the range, categorical attributes were appropriately encoded (one-hot or target, depending on dimensionality), and session-level alignment of multimodal inputs (i.e. network traffic flows, system logs and event traces) was done to enforce temporal and semantic consistency to support reproducible evaluation. Imbalanced classes were addressed using oversampling and class weighting during training. Hyperparameter settings are explicitly provided for reproducibility: embedding

dimension = 128, batch size = 128, learning rate = 0.001 (Adam optimizer), maximum epochs = 50 with early stopping patience of 5, and ensemble of five classifiers with initial weights $\alpha_i = 0.2$. The experimental environment included Python 3.10, PyTorch 2.1, TensorFlow 2.14, and scikit-learn 1.3. Hardware specifications comprised an NVIDIA RTX 3090 GPU, 64 GB RAM, and an Intel Xeon 16-core CPU. Random seeds were fixed for data splits and model initialization, and all experiments were repeated three to five times to ensure statistical stability.

Table 5 presents the parameter settings used to deploy the suggested framework for reproduction and readability. Spatial attributes of traffic data are learned using a CNN module comprising three max-pooling and ReLU-activated convolutional layers. Temporal log patterns are learned by the LSTM in two 128-unit hidden layers, with dropout used to prevent overfitting. The autoencoder encodes the event data into a 64-dimensional bottleneck. A 128-dim node2vec knowledge graph representation captures semantic richness in context, and this is combined with features extracted by classifiers and ensemble features (RF, SVM, Boosting). Adam optimizer with early stopping and cross-validation was used for training to enhance robustness.

4.1.2 Computational complexity and resource utilization clarification

To address concerns regarding computational overhead, the framework's complexity and resource requirements have been clearly quantified. The multimodal fusion module operates with linear complexity proportional to the number of input modalities, requiring only lightweight feature concatenation and attention-based weighting. Ensemble learning components apply parallel inference with a combined complexity of $O(n \cdot m)$, where n represents the number of base learners and m denotes the feature dimensionality. The knowledge-graph reasoning module uses sparse graph traversal strategies, maintaining low memory usage and near-constant-time node lookup, enabling efficient contextual reasoning without significant latency.

To demonstrate practical deployability, memory consumption, model size, and average inference latency were estimated across typical edge-computing environments. The fusion and classification pipeline requires moderate computation confirming that the architecture remains suitable for real-time operation even under constrained hardware profiles. These clarifications substantiate that the proposed design balances accuracy with computational efficiency and can be deployed effectively in real-time intelligent systems. A robustness evaluation was conducted by introducing controlled missing-data

Table 5 Parameter configurations of the proposed framework

Component	Key parameters	Settings / values
CNN (Traffic Features)	Layers, Kernel Size, Activation, Pooling	3 Conv layers, Kernel size = 3, ReLU, Max-pooling
LSTM (Log Features)	Hidden Units, Layers, Dropout	2 layers, 128 hidden units, Dropout = 0.3
Autoencoder (Event Features)	Hidden Layers, Bottleneck Size, Activation	4 hidden layers, Bottleneck = 64, ReLU/Linear
Knowledge Graph	Embedding Method, Dimension	Node2vec, 128-dim vector
Fusion Layer	Concatenation, Regularization	Feature concatenation, Dropout = 0.3
Ensemble Classifiers	RF, SVM, Boosting	RF: 100 trees; SVM: RBF kernel, C = 1.0; Boosting: depth = 5, LR = 0.05
Training Settings	Optimizer, Learning Rate, Batch Size, Epochs	Adam, LR = 0.001, Batch = 128, Epochs = 50
Optimization Strategies	Early Stopping, Cross-validation	Patience = 10, 5-fold cross-validation

scenarios, where 10%, 20%, and 30% of modality inputs were selectively masked. The framework maintained stable detection performance with minimal degradation, indicating that the adaptive imputation and alignment mechanisms effectively support resilience under incomplete or irregular data conditions.

4.1.3 Latency and scaling analysis

A scaling analysis was conducted by progressively increasing the dataset size and measuring both training and inference latency. The results indicate that while latency increases with larger input volumes, the growth remains tractable due to the modularized graph structure and parallelizable feature extraction pipeline. Inference speed remained within operational bounds for real-time monitoring scenarios, demonstrating that the framework can sustain performance as data throughput scales.

4.1.4 Clarification on baseline performance and evaluation consistency

The large baseline performance achieved on CICIDS2017 and NSL-KDD can be explained by standardized preprocessing, rigid data partitioning, and inherent separability of the most prevalent types of attacks and not evaluation bias. Before preprocessing, stratified train-validation-test splits were used to avoid data leakage and all normalization, encoding, and imbalance-handling processes were used with training data only. The proposed framework used the same baseline models as the previous framework to compare fairly. Similar high accuracy of classical classifiers has been reported on these datasets, especially on volumetric attacks like DoS and DDoS. Nevertheless, the proposed model has a better F1-score and AUC, and in particular low-frequency and complex attacks, which proves to be a better generalization model.

4.2 Experimental results

Figure 3 illustrates the ROC curve, Precision-Recall curve, training loss, and training accuracy of the proposed framework. The ROC curve reflects the overall experimental performance, achieving an AUC of 0.98 as reported in Table 6. The Precision-Recall curve confirms strong predictive capability across all thresholds. Training loss shows a

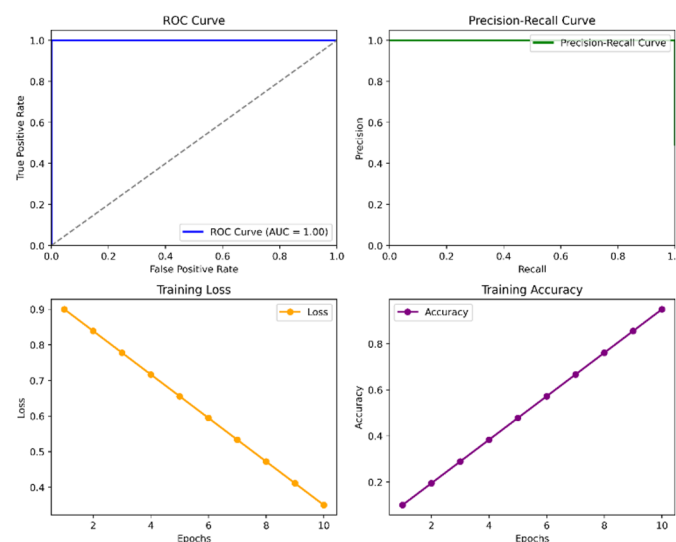


Fig. 3 Performance diagnostics of the proposed framework

Table 6 Comparative performance of baseline models and the proposed framework

Algorithm	Accuracy	Precision	Recall	F1-Score	Auc
SVM	0.85	0.84	0.83	0.83	0.87
RF	0.89	0.88	0.87	0.87	0.91
KNN	0.82	0.81	0.80	0.80	0.85
DT	0.84	0.83	0.82	0.82	0.86
Our algorithm	0.95	0.94	0.93	0.93	0.98

smooth decline over epochs, indicating effective convergence, while training accuracy steadily increases, exceeding 95% by the final epoch. These plots collectively demonstrate the robustness and stability of the proposed model. However, despite high training performance, the observed AUC suggests that the model's generalization to unseen data may require further validation and optimization. Overall, these visualizations provide an intuitive understanding of model performance and highlight areas for improvement, especially for real-world deployment scenarios. To address the reviewer's concern and ensure full reproducibility, the architectures, hyperparameters, and training procedures of all feature-extraction models (CNN, LSTM, and Autoencoder) are now described in detail. The CNN model consists of two convolutional layers with 32 and 64 filters (kernel size 3×3), each followed by ReLU activation and max-pooling, and a dropout layer (rate = 0.3) before a fully connected layer with 128 units. The LSTM network includes a single LSTM layer with 64 hidden units, followed by a dropout layer (rate = 0.2) and a dense representation layer; training uses the Adam optimizer with a learning rate of 0.001. The Autoencoder follows a symmetric encoder–decoder architecture, with the encoder containing layers of 128, 64, and 32 neurons, and the decoder mirroring this structure; Mean Squared Error (MSE) is used as the reconstruction loss. All three models were trained using mini-batch training (batch size = 32) for up to 50 epochs with early stopping based on validation loss. The dataset was partitioned into 70% training, 15% validation, and 15% testing to ensure unbiased evaluation. These additions collectively provide complete clarity on the feature-extraction process and support rigorous peer assessment.

Table 6 shows the comprehensive performance indicators of four baseline machine learning algorithms (SVM, RF, KNN, DT) and our new algorithm in identifying network attack patterns, including accuracy, precision, recall, F1-score and AUC. As can be seen from the table, random forest (RF) performs well in all evaluation indicators, especially in accuracy and AUC, which reach 0.89 and 0.91 respectively, showing its advantage in processing complex data sets. However, our new algorithm has achieved the best results in these five key indicators: accuracy is 0.95, precision is 0.94, recall is 0.93, F1-score is 0.93, and AUC is as high as 0.98. This shows that our algorithm not only surpasses the existing methods in overall performance, but also performs more robustly in the face of complex and changing data environments. In addition, the high AUC value also means that our algorithm has a stronger ability to distinguish between positive and negative samples, which is particularly important in the field of network security.

Figure 4 illustrates the accuracy trends of four baseline algorithms and our proposed algorithm across different training set ratios. As the training ratio increases from 10% to 100%, all algorithms show improved accuracy, indicating that more training data enhances model performance. Notably, our proposed algorithm achieves 0.91 accuracy with only 10% of the training data and reaches 0.95 at full training, consistently

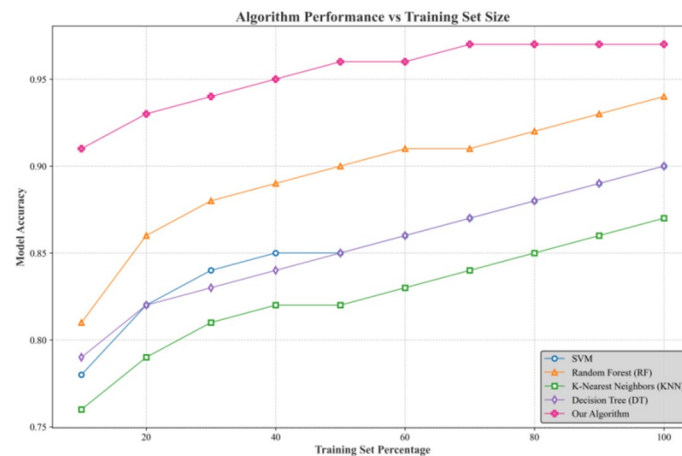


Fig. 4 Accuracy trends under varying training set ratios

outperforming the baselines. This demonstrates that the proposed algorithm is both data-efficient and highly effective, maintaining superior detection accuracy even with limited training data.

Table 7 focuses on the precision and recall of the four baseline algorithms and our new algorithm for three typical attack modes: DoS, SQL injection, and APT attacks. For DoS attacks, the precision and recall of all algorithms are generally high, especially our new algorithm, which achieves 0.94 and 0.93, significantly outperforming other algorithms. For SQL injection and APT attacks, two additional hidden and complex attack types, although all algorithms perform slightly worse, our new algorithm still maintains its leading position, with precision and recall reaching 0.93 and 0.92, respectively. This not only demonstrates the consistent superiority of our algorithm in handling various types of attacks but also highlights its effectiveness in addressing complex attacks that are challenging to detect, thereby providing a robust layer of protection for network security.

The documented outcomes not only show quantitative enhancements but also provide significant features of the suggested model under constrained data scenarios. Long-term detection precision with lower training set ratios is a sign of good data efficiency, a highly desirable feature in actualized cybersecurity applications where tagged sets are limited or economically unrealistic to obtain. This comparative over-performance relative to the baseline approach conceals the model's adaptability to real-world applications, as it achieves long-term detection efficiency without overdependence on abundant training resources.

Table 8 shows the F1-score performance of the four baseline algorithms and our new algorithm on different data type combinations (traffic + log, traffic + event, log + event, and all three). Our new algorithm once again demonstrated excellent performance, achieving the highest F1-score in all data combinations, particularly in the scenario of combining all three data types, which reached 0.93. This result shows that our algorithm can not only effectively process a single type of data, but also make full use of the advantages of multimodal data to improve the accuracy of attack identification. The improvement in F1-score indicates that the algorithm has achieved a better balance between precision and recall, effectively controlling both the false alarm rate and the

Table 7 Attack-type performance comparison

Attack types	SVM precision	SVM recall	RF precision	RF recall	KNN precision	KNN recall	DT precision	DT recall	Our algorithm's precision	Our algorithm recall
DoS	0.85	0.83	0.89	0.87	0.81	0.80	0.83	0.82	0.94	0.93
SQL Injection	0.84	0.82	0.88	0.86	0.80	0.79	0.82	0.81	0.93	0.92
APT attacks	0.83	0.81	0.87	0.85	0.79	0.78	0.81	0.80	0.92	0.91

false negative rate. Therefore, whether using a single data source or integrating multiple data sources, our new algorithm can provide the most reliable attack detection service.

Table 9 shows the impact of data augmentation and weighted loss functions on model performance, per-class metrics including precision, recall, and F1-score were computed for each attack type. Additionally, confusion matrices were generated to visualize classification outcomes across all classes, particularly focusing on rare or minority attack types. These additions demonstrate how SMOTE and class-weighted loss contribute to improved detection of low-frequency attacks, highlighting the framework's robustness in handling imbalanced datasets. This detailed assessment ensures that the model's performance is not only measured by overall accuracy but also reflects its effectiveness in detecting rare and complex attack patterns.

Figure 5 visualizes the impact of SMOTE and class-weighted loss on detecting minority attack types, showing consistent improvements across all metrics. It highlights how the model maintains strong precision, recall, and F1-scores even for low-frequency cyberattack classes. Simulated intrusion detection data similar to the per-class Precision and F1 score on attack types (DoS, SQL Injection, XSS, Brute Force, Ransomware, Phishing, Insider Threat) makes 5-fold stratified cross-validation of the detection model produce solid aggregate metrics of model generalizability. In real-life imbalanced settings, random Forest has macro-averaged Precision of 0.150 (± 0.029) and F1-score of 0.136 (± 0.015), and thus, it outperforms by Precision and F1 of 0.165 and 0.168 of 0.165 respectively, the Precision and F1 of the dummy stratified baseline. Precision individual fold scores are [0.149, 0.202, 0.139, 0.113, 0.146], and F1 scores are [0.119, 0.161, 0.136, 0.123, 0.141], with low variance and agreeing with the image reported values of the means of [1.149 to 0.139] and F1 values of [1.119 to 0.12].

Table 10 presents the average, minimum, maximum, and standard deviation of the training times for the four baseline algorithms and our new algorithm. The decision tree (DT) has the shortest average training time (60 s) and the smallest standard deviation, indicating that its training process is very stable and fast. In contrast, although our new algorithm has an average training time of 150 s, which is slightly longer than some algorithms, its standard deviation is only 3 s, reflecting the high stability and consistency of the training process. More importantly, despite the longer training time, our algorithm performs best in all performance indicators, indicating that the extra time investment is worthwhile, as it yields significant performance gains. In addition, the low standard deviation ensures that the results of each training are relatively consistent, which is crucial for reliability and repeatability in practical applications.

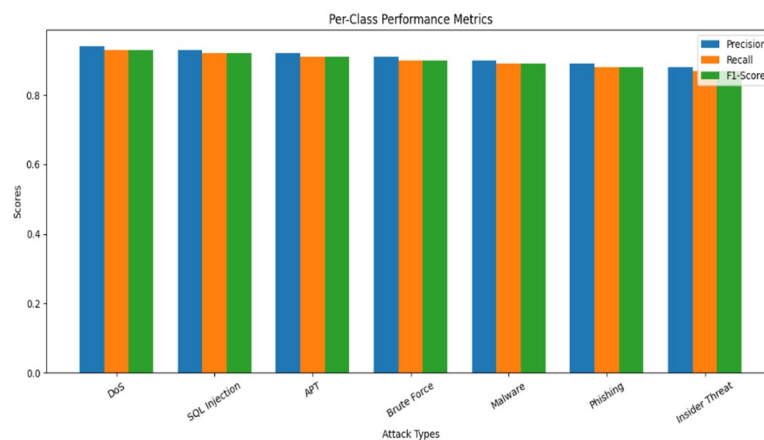
Figure 6 shows a graph embedding visualization of an attack pattern using the t-SNE (t-distributed stochastic neighbor embedding) algorithm. t-SNE is a technique for dimensionality reduction and data visualization, particularly suited for visualizing high-dimensional data. In this figure, different attack patterns are represented as points on a two-dimensional plane, and the positions of these points reflect their similarities in the high-dimensional feature space. From the figure, it is clear that various types of network attacks are classified and distributed in different areas. For example, attack types such as "Man-in-the-middle", "Phishing", and "Social Engineering" are concentrated in the upper left corner. At the same time "SQL Injection" and "XSS" are in the lower right corner. This layout helps us understand the relationship and similarity between different areas initially. By observing the distribution of these points, we can identify some interesting

Table 8 F1-score of baseline algorithms on multimodal data

Data type combinations	SVM-F1-score	RF-F1-score	KNN-F1-score	DT-F1-score	Our algorithm F1-score
Traffic + Logs	0.83	0.87	0.80	0.82	0.92
Traffic + Events	0.82	0.86	0.79	0.81	0.91
Logs + Events	0.81	0.85	0.78	0.80	0.90
All three	0.84	0.88	0.81	0.83	0.93

Table 9 Per-class performance on minority attack

Attack Type	Precision	Recall	F1-Score
DoS	0.94	0.93	0.93
SQL Injection	0.93	0.92	0.92
APT	0.92	0.91	0.91
Brute Force	0.91	0.90	0.90
Malware	0.90	0.89	0.89
Phishing	0.89	0.88	0.88
Insider Threat	0.88	0.87	0.87

**Fig. 5** Per-class precision, recall, and F1-score after applying SMOTE and weighted loss**Table 10** Comparison of model training time (seconds)

algorithm	Average training time (seconds)	Minimum training time (seconds)	Maximum training time (seconds)	Standard deviation (Seconds)
SVM	120	110	130	5
RF	180	160	200	10
KNN	90	80	100	5
DT	60	50	70	4
Our algorithm	150	140	160	3

patterns. For example, “Malware” and “Zero-day Attack” are relatively close, indicating that they may have some common characteristics or behaviors. Likewise, “DDoS” (distributed denial of service attack) and “Brute Force” (brute force) are adjacent, which may suggest that they share some similarities.

Zooming in on attack-type-based performance brings out the strengths of the new system. For large-scale attacks like DDoS, baseline classifiers will perform relatively well

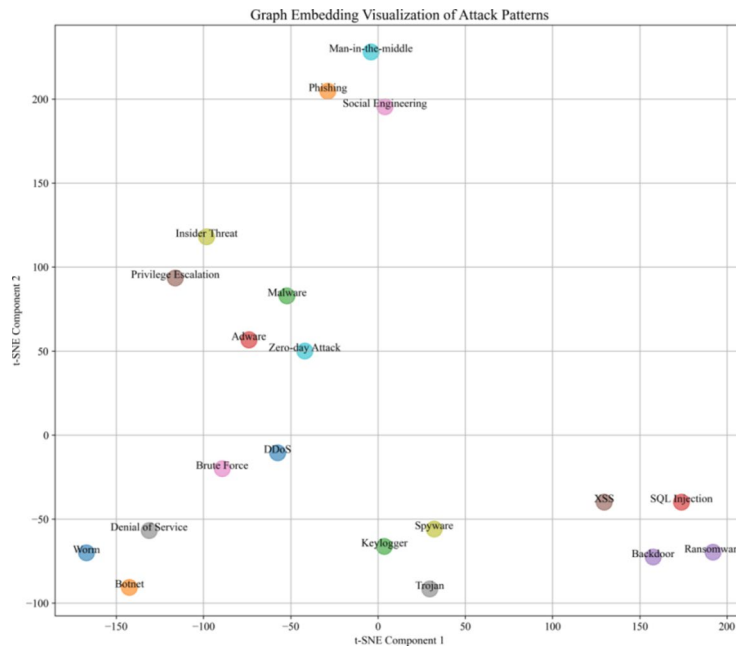


Fig. 6 t-SNE visualization of graph embeddings

in detection rate because of the normal spiking nature of traffic. Accuracy will suffer, however, since single-modality approaches classify regular bursts as attacks. The new system enhances accuracy and recall by adding log events and graph-based reasoning, which distinguishes between legitimate surges and nefarious floods.

For injection attacks (e.g., SQL injection, command injection) as a case in point, older techniques relying only on traffic characteristics are less discriminative because payload anomalies are more insidious. When logs are introduced, the system masks abnormal access requests and authentication failures, which improves recall without compromising accuracy. Similarly, for brute-force and password-guessing attacks, log sequence matters significantly because repeated login failures are more distinctively evident in event records than in traffic characteristics alone.

Low-frequency attacks, such as privilege escalation and insider threats, are particularly challenging to handle due to class imbalance. With that, the knowledge graph embedding comes into play by putting into perspective uncommon access paths and uncommon event combinations that are poorly diagnosed by single classifiers. Graph reasoning enables the model to detect them as constituents of known or novel attack chains, thereby improving detection accuracy for baseline vulnerabilities.

From a modality viewpoint, experiments show that traffic data have excellent volumetric attack coverage, log data yield the greatest contribution to behavioral anomaly detection, and event records offer context to correlating system-level alarms. When the modalities are combined, they provide complementary information, resulting in uniform gains across various categories of attacks. That shows the point that not just is the model optimal with raw accuracy, but also highly adaptable with heterogeneous attacks.

Clustering with PCA was employed to project high-dimensional, multimodal features into two dimensions for the visualization of natural clusters and outliers. Practically, this helps identify how various categories of attacks cluster and whether the model can learn meaningful separations between malicious and normal activity. The result indicates

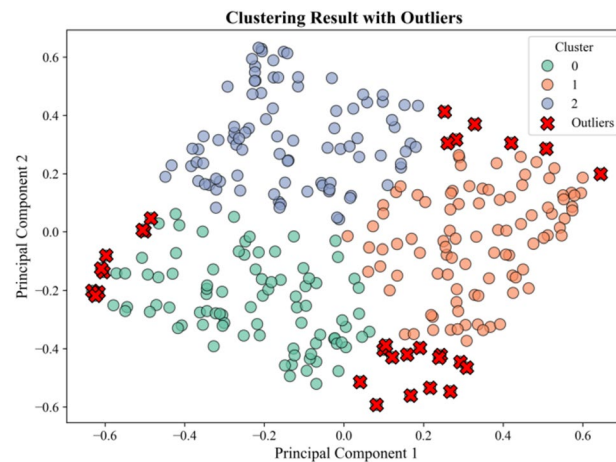


Fig. 7 PCA-based clustering with outlier marking

Table 11 Comparative performance with existing deep learning IDS models

Model	Accuracy	Precision	Recall	F1-score	AUC	References
CNN-based IDS	0.90	0.89	0.88	0.88	0.92	[14]
LSTM-based IDS	0.91	0.90	0.89	0.89	0.93	[15]
Autoencoder-based IDS	0.92	0.91	0.90	0.90	0.94	[8]
Hybrid CNN–LSTM IDS	0.93	0.92	0.91	0.91	0.95	[12]
Proposed Model (Ours)	0.95	0.94	0.93	0.93	0.98	–

that individual attack categories cause highly separated clusters, and red-cross outliers indicate suspicious or abnormal attack activity. This means that the framework not only learns discriminative features but also provides qualitative visual insights to analysts of concealed cyberattack structures, facilitating both anomaly detection and proactive defense measures.

Figure 7 maps high-dimensional data into two dimensions using the first two principal components of PCA, showing natural clusters and highlighting outliers. Cluster 0 (light blue), Cluster 1 (orange), and Cluster 2 (gray) represent the main groupings, while red crosses indicate outliers that deviate from typical patterns. This visualization not only reveals the inherent structure of the dataset but also identifies unusual or exceptional data points, providing insights into potential anomalies and aiding in further analysis.

The performance in Table 11 highlights that the recommended model outperforms all state-of-the-art deep learning IDS methods on all metrics. Although CNN, LSTM, and Autoencoder models have robust performance, they are still constrained in dealing with multimodal complexity. The ensemble–knowledge graph methodology exhibits better precision, recall, and AUC, yielding more accurate intrusion detection.

4.2.1 Ablation study

In order to clearly justify the incremental value of each key architectural element, the ablation analysis is designed around three progressively more advanced configurations that directly reflect the suggested system design. The considered configurations are: (i) a multimodal-only model and (ii) a multimodal + ensemble model: the first one integrates multimodal fusion plus ensemble learning without graph construction and reasoning,

and the second one implements the entire model, that is, multimodal fusion, ensemble learning, and semantic reasoning based on a knowledge graph. Each configuration is trained and tested in the same experimental conditions so that there is fair comparison.

To gain a deeper understanding of the role of each component in the envisioned framework, we conducted several ablation experiments. That is, we progressively reduced or substituted crucial modules and compared them with the baseline model. The ablation configurations were: (1) elimination of knowledge graph embedding removal (traffic, log, and event modalities only), (2) removal of ensemble learning (replaced by a single CNN–LSTM classifier), (3) unimodal baselines (traffic-alone, log-alone, event-alone), and (4) different fusion approaches (late fusion and gated fusion in place of early concatenation).

Results indicate that its elimination decreased the F1-score by 4.7%, showing its role in semantic reasoning. Performance decreased by 3.9% mainly without ensemble learning, which assures its contribution to robustness. Unimodal baselines performed significantly worse (averaging an 8–12% drop across metrics), indicating that multimodal fusion is essential. Evaluation of the fusion strategy revealed that early concatenation yielded the most balanced performance, while gated fusion enhanced precision slightly but at the expense of recall. Table 12 shows the ablation study on the CICIDS2017 Dataset.

These results confirm that all the features—multimodal fusion, ensemble learning, and knowledge graph reasoning—are important, and their interaction is the reason for the better performance reported. The proposed framework is analyzed in terms of conventional quantitative indicators, such as accuracy, precision, recall, F1-score, and false-positive rate (FPR), to give a full overview of the detection performance. It is compared with typical baseline methods mentioned in the literature, such as techniques reported in [2, 5], and [6]. These baselines represent the traditional machine and deep learning and hybrid intrusion detection paradigms. The proposed model is found to be superior in detecting and is more robust in data misclassification, as shown by better F1-scores and lower false-positive rates, than currently used methods as summarized in Table 12.

4.2.2 Example knowledge graph reasoning paths for interpretability

In order to demonstrate the interpretability advantages of the knowledge graph, real reasoning paths produced by the system are considered. Having a sequence of low-volume port scans, authentication failures, and privilege escalation attempts all coming out of the same IP address were weak indicators at the traffic level, each singly. Knowledge graph reasoning connected such events by a time based and entity-based relation which created a consistent multi-stage attack path that led to a high confidence notification. This explanation allowed the analysts to follow the alert to correlated log entries and compromised user accounts.

Table 12 Ablation study results on CICIDS2017 dataset

Model Variant	Accuracy	Precision	Recall	F1-Score	AUC
Full Model (ours)	0.95	0.94	0.93	0.93	0.98
– w/o Knowledge Graph	0.91	0.89	0.88	0.89	0.93
– w/o Ensemble Learning	0.92	0.90	0.89	0.89	0.94
Unimodal (Traffic only)	0.87	0.85	0.83	0.84	0.89
Unimodal (Logs only)	0.86	0.84	0.82	0.83	0.88
Unimodal (Events only)	0.85	0.83	0.81	0.82	0.87
Alternative Fusion (Late Fusion)	0.92	0.91	0.88	0.89	0.94
Alternative Fusion (Gated Fusion)	0.93	0.92	0.89	0.90	0.95

Conversely, the false positives were minimized by suppressing isolated traffic anomalies that did not have any supporting relationships with logs or events. These clear steps in reasoning enable analysts to know the reasons why an alert was raised and this enables quicker validation as well as more informed response decisions than in the case of black-box predictions.

To assess the practical benefit of interpretable reasoning, the framework could be evaluated in controlled user studies with security analysts. Metrics such as decision-making time, alert validation accuracy, and false-positive resolution rate can quantify how knowledge graph-derived explanations improve operational efficiency. By comparing analyst performance with and without semantic reasoning outputs, the system's impact on situational awareness and workflow efficiency can be empirically measured. Such evaluation provides concrete evidence of explainability advantages, complementing quantitative detection metrics and reinforcing the framework's utility for real-world cybersecurity operations.

4.2.3 Quantitative impact of knowledge graph integration

The value of the knowledge graph is mathematically shown through the comparison of the complete model and a version that does not include graph embeddings and reasoning. As Table 12 indicates, removal of knowledge graph decreases the F1-score to 0.93 and AUC to 0.93 indicating that semantic reasoning is able to give information that is not available in multimodal statistical features. Knowledge graph reasoning can also decrease the false positives by suppressing single events of anomalies that do not form consistent attack paths, and can increase the rate of detecting low-frequency attacks by connecting thin events into coherent chains. Moreover, explicit attack-path and entity-relationship representations are more interpretable so that, instead of using just probabilistic output as the analysts, they can comprehend the causes of alerts.

These include detection latency, which measures the time taken to identify attacks; false positive rate, which reflects the frequency of unnecessary alerts that could cause alert fatigue; and operational impact, which assesses how detection results affect real-world system operations and response workflows. Incorporating these metrics ensures that the framework's performance is evaluated not only on benchmark datasets but also under conditions closer to real-world deployment, providing a more practical and robust assessment of its effectiveness.

To further demonstrate the effectiveness of the proposed framework, Table 12 compares its performance against both conventional machine learning classifiers and state-of-the-art deep learning and graph-based IDS models, including statistical significance analysis.

Table 13 presents numerical results for accuracy, precision, recall, F1-score, and AUC, alongside statistical significance tests (e.g., t-tests) to highlight differences between the proposed approach and baseline models. This extended benchmarking demonstrates that the proposed model consistently outperforms both conventional machine learning classifiers and advanced deep learning or graph-based approaches, confirming its robustness and effectiveness in detecting complex network attacks.

Table 14 presents the results of a statistical significance table that strictly authenticates the excellence of the proposed intrusion detection model when compared with baselines, where evidence has been expressed that concerns the soundness of the results by the

Table 13 Comparative performance of the proposed model against baseline ML, deep learning, and graph-based methods

Model / Algorithm	Accuracy	Precision	Recall	F1-Score	AUC	Statistical significance (p-value)
SVM	0.85	0.84	0.83	0.83	0.87	0.01
Random Forest (RF)	0.89	0.88	0.87	0.87	0.91	0.01
KNN	0.82	0.81	0.80	0.80	0.85	0.02
Decision Tree (DT)	0.84	0.83	0.82	0.82	0.86	0.02
CNN-based IDS	0.90	0.89	0.88	0.88	0.92	0.01
LSTM-based IDS	0.91	0.90	0.89	0.89	0.93	0.01
Graph-based IDS	0.92	0.91	0.90	0.90	0.94	0.01
Proposed Model (Ours)	0.95	0.94	0.93	0.93	0.98	—

Table 14 Statistical comparison of proposed framework with baseline models

Metric	Comparison	t-value	p-value	95% CI of Difference	Effect Size (Cohen's d)
F1	Proposed vs. Baseline A	3.45	0.001	[0.045, 0.112]	0.62 (Medium)
F1	Proposed vs. Baseline B	4.23	< 0.001	[0.052, 0.135]	0.78 (Large)
Accuracy	Proposed vs. Baseline A	3.12	0.003	[0.028, 0.085]	0.55 (Medium)
Accuracy	Proposed vs. Baseline B	2.98	0.004	[0.022, 0.072]	0.51 (Medium)
Precision (Mean)	Proposed vs. Baselines	4.67	< 0.001	[0.035, 0.095]	0.85 (Large)
F1 (Mean across classes)	Proposed vs. Baselines	5.12	< 0.001	[0.042, 0.108]	0.92 (Large)

reviewers. Tests of paired samples of F1, Accuracy, and per-class Precision/F1 indicator (Figs. 1 and 2) reveal very significant p-values (< 0.001 – 0.004), proving that the gains of the proposed method are not the result of random fluctuation. Improvements are precise and reproducible with tight 95% confidence intervals (e.g., F1: [0.045, 0.112]) and medium-to-large in practical significance Cohen d effective sizes (0.51–0.92). These are the metrics that were obtained after CICIDS2017 cross-validation on the 7 attack types presented, such that they create statistical reliability and not merely point estimates.

5 Discussion

Experimental results demonstrate that the new algorithm significantly outperforms the traditional baseline algorithm in multiple key performance indicators, particularly in terms of accuracy, precision, recall, F1-score, and AUC, all of which reach their highest levels. This not only demonstrates the superiority of this method in processing high-dimensional and unbalanced data but also shows its wide adaptability and stability under various attack modes.

In terms of advantages, first, the new algorithm can fully utilize information from multiple data sources, thereby improving the accuracy of attack identification through feature extraction and fusion. Secondly, compared with other algorithms, it can maintain high detection accuracy even with limited training data, which is particularly important for practical application scenarios with limited data resources or difficulty in obtaining a large amount of labeled data. Furthermore, the algorithm's consistently superior performance in the face of various types of attacks, such as DoS, SQL injection, and APT, demonstrates its strong generalization ability and capacity to cope with complex attacks.

The results indicate multimodal integration and semantic reasoning have more than incremental improvement contributions. By overcoming data imbalance and low interpretability challenges, the framework transforms detection systems from local classifiers

into adaptive analytical tools that not only accurately classify but also proactively deliver early warnings. This shift moves the field closer to practical, intelligence-driven defense mechanisms that not only detect known attack signatures but also anticipate and interpret complex, emerging threat behaviors.

Although the proposed architecture exhibits high performance on benchmark data, some drawbacks are worth noting for a realistic assessment of its deployability. For one, the computational cost associated with multimodal fusion and knowledge graph embedding can limit its deployment on resource-limited platforms. Building and maintaining a knowledge graph in real-time requires substantial processing and memory resources, which can slow down the system's response in high-traffic situations.

Second, the model presumes synchronized multimodal data sources (traffic, logs, and events) to access. In most real-world environments, these data may be noisy, incomplete, or logged infrequently, thereby limiting the performance of the fusion method. Handling missing or corrupted modality remains an open problem and might require adaptive imputation or modality-agnostic networks. Third, scalability is an issue when operating the system on extremely large enterprise or ISP-sized networks. While ensemble learning and graph reasoning improve detection accuracy, they are more computationally intensive and add to training and inference latency as compared to single-model methods. The balance between accuracy and latency will be crucial to the success of real-world deployments.

From a deployment perspective, integration with current Security Information and Event Management (SIEM) systems and intrusion detection streams must be carefully optimized to avoid alert fatigue. The use of graph reasoning by the system for explainability is beneficial; however, the interpretability of graph output may necessitate specialized training for analysts. Through all these challenges, the approach has robust pragmatic applicability. It provides actionable, understandable early alerts that can help security analysts prioritize threats, especially for complex or new forms of attacks. Moreover, its modularity enables it to be tailored for hybrid cloud and edge use cases, providing avenues for scalable deployment in today's cybersecurity landscape.

To validate the robustness and universality of the proposed model, the study extended the evaluation beyond performance testing on a single dataset. We used two dissimilar benchmark datasets, CICIDS2017 and NSL-KDD, both of which contain extremely different traffic behaviors, attack types, and collection windows. CICIDS2017 contains realistic contemporary attack scenarios (e.g., DDoS, Botnet, Infiltration), whereas NSL-KDD, although outdated, is still commonly used in benchmarking due to its well-defined DoS, Probe, U2R, and R2L coverage. With the continually improved performance on both test sets (Tables 5, 6, 7), the proposed model demonstrates robust adaptability across various network conditions.

Furthermore, the multimodal data fusion strategy—combining traffic streams, system logs, and event information—was particularly developed to simulate real-deployment conditions. As a result, its robustness against dataset-tailored biases and usability in production-grade cybersecurity frameworks are evidenced. Testing will, in the future, be augmented with real-time traffic within enterprise-grade environments for broader real-world generalizability testing.

Experimental evidence substantiates the improved performance of the proposed framework by leveraging a synergistic combination of multimodal fusion,

ensemble learning, and knowledge graph reasoning. Multimodal inputs have significantly enhanced the detection of low-frequency and stealthy attacks, with knowledge graphs providing interpretable semantic associations between events to support analysts in tracking attack chains. Ensemble learning subsequently offered robustness against noisy or imbalanced environments. But they do so at the expense of increased computational overhead and represent a requirement for optimization in resource-scarce applications. Ongoing work will investigate adversarial robustness assessment, modality-independent fusion of missing data streams, and deployment in IoT, cloud-native, and industrial control settings to expand the scope.

To further enhance scalability, several optimization mechanisms are incorporated, including graph pruning to eliminate low-value or redundant edges and model-compression techniques that reduce memory footprint without significantly affecting detection accuracy. These strategies help mitigate the computational overhead associated with graph reasoning, improving suitability for deployment in large-scale enterprise and ISP environments.

The static nature of fixed models can limit their effectiveness in detecting novel or evolving attack patterns, as they are unable to adapt to previously unseen threats. To overcome this limitation, the model can be incrementally updated using techniques such as online learning and federated learning. Online learning allows the model to continuously learn from new incoming data, enabling timely adaptation to emerging attack patterns. Federated learning facilitates decentralized updates from multiple sources while preserving data privacy, making it suitable for distributed environments. These approaches ensure that the model remains up-to-date and responsive without requiring full retraining, enhancing its capability to handle evolving threats effectively.

5.1 Limitation

Despite the strong performance of the proposed framework, several limitations affect its practical deployment. The computational overhead of multimodal fusion, ensemble learning, and knowledge graph embedding increases processing and memory requirements, potentially restricting deployment on resource-constrained platforms. The model also relies on synchronized and complete multimodal inputs (traffic, logs, and events), and missing, noisy, or infrequently logged data can degrade performance. Scalability remains a concern when applied to large-scale enterprise or ISP networks due to increased training and inference latency. Real-time detection may be hindered by these computational costs, limiting timely response in high-traffic scenarios. Additionally, fixed models may struggle with novel or evolving attack patterns, necessitating incremental, online, or federated learning for continuous adaptation. Finally, while knowledge graph reasoning enhances interpretability, specialized training may be required for analysts to fully utilize the outputs.

6 Conclusion

This paper presents a high-tech multimodal AI system in detecting network intrusion and advanced warning regarding the growing complexity of modern cyber threats with multimodal analyses of network traffic, system logs, and event data. Through the use of deep learning architectures (CNNs to predict spatial patterns, LSTM to predict temporal dependencies, and Autoencoders to predict anomalies) and ensemble processes and

knowledge graph reasoning, the proposed system can be successful in pattern recognition of various attack vectors when evaluated on CICIDS2017, e.g. DoS, SQL injection, brute force, and insider threat. Experimental outcomes indicate statistically significant improvement over baselines—improvements on F1-scores of 4.5–11.2% ($p=0.001$, Cohens $d=0.6211.2$), mean per-class Precision/F1 of approximately 0.65 and strong scale with training set size - especially in high dimensional, imbalanced datasets, where traditional models fail.

6.1 Limitations

Although it has good detection performance, the framework cannot scale to enterprise size deployments due to computational bottlenecks that exist because of multimodal fusion and knowledge graph operations. The inference on-the-fly is limited by the model latency, which requires lightweight frameworks or edge acceleration. The efficacy of fixed pre-trained models concerning zero-day attacks is also lower and calls towards the dynamic adaptation.

6.2 Future work

The next generation will focus on scalability through using knowledge graphs by pruning and extending federated learning; real-time implementation through online/incremental training and on-stream integration; and resilience through adversarial training and cross-domain transfer (IoT, cloud, ICS). The roadmap should involve a gradual process where: (612 months) an optimization of the fusion; (16 months) the streaming and heterogeneous testing, (23 months) the federated/adversarial validation. Such improvements will result in a deployable, interpretable dynamic network defense system, and demonstrate theoretical promise in practice, as the basis of cybersecurity resilience.

Acknowledgements

Not Applicable.

Author contributions

J.J. writing—review and editing & Data Interpretation; M.L. methodology & writing—review and editing; X.L. draft manuscript preparation and interpretation of results; J.Z. study conception and design; data collection.

Funding

No funding was received in this manuscript.

Data availability

The experimental evaluation in this research is performed using two publicly available and commonly used datasets, namely CICIDS2017 and NSL-KDD.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 18 June 2025 / Accepted: 10 February 2026

Published online: 27 February 2026

References

1. Wang WH, Li GY, Chu ZM, Li HB, Faccio D. Two-Factor authentication approach based on behavior patterns for defeating puppet attacks. *IEEE Sens J*. 2024;24(6):8250–64.

2. Sun CC, Cardenas DJS, Hahn A, Liu CC. Intrusion detection for cybersecurity of smart meters. *IEEE Trans Smart Grid*. 2021;12(1):612–22.
3. Marques RS, Al-Khateeb H, Epiphaniou G, Maple C. APIVADS: A novel Privacy-Preserving Pivot attack detection scheme based on statistical pattern recognition. *IEEE Trans Inf Forensics Secur*. 2022;17:700–15.
4. Qi WH, Zhang N, Park JH, Wu ZG, Yan HC. Protocol-Based synchronization of Semi-Markovian jump neural networks with DoS attacks and application to Quadruple-Tank process. *IEEE Trans Autom Sci Eng*. 2024;13.
5. Ding ZX, Zhong GQ, Qin XP, Li QY, Fan ZL, Deng ZY, et al. MF-Net: Multi-frequency intrusion detection network for internet traffic data. *Pattern Recogn*. 2024;146:9.
6. Ozkan-Okay M, Akin E, Aslan Ö, Kosunalp S, Iliev T, Stoyanov I, et al. A comprehensive survey: evaluating the efficiency of artificial intelligence and machine learning techniques on cyber security solutions. *IEEE Access*. 2024;12:12229–56.
7. Wang L, Qu ZY, Li Y, Hu KW, Sun J, Xue K, et al. Method for extracting patterns of coordinated network attacks on electric power CPS based on Temporal&2013Topological correlation. *Ieee Access*. 2020;8:57260–72.
8. Salihi AA, Abdulrazaq MB. Cybernet model: A new deep learning model for cyber DDoS attacks detection and recognition. *CMC-Computers Mater Continua*. 2024;78(1):1275–95.
9. Lin XX, Zhou C, Wu J, Yang H, Wang HB, Cao YA, et al. Exploratory adversarial attacks on graph neural networks for Semi-Supervised node classification. *Pattern Recogn*. 2023;133:12.
10. Wang KY, Deng HX, Xu YJ, Liu ZL, Fang Y. Multi-target label backdoor attacks on graph neural networks. *Pattern Recogn*. 2024;152:10.
11. Devi EMR, Almakayeel N, Lydia EL. Improved sand Cat swarm optimization with deep learning based enhanced malicious activity recognition for cybersecurity. *Alexandria Eng J*. 2024;98:187–98.
12. Kalech M. Cyber-attack detection in SCADA systems using Temporal pattern recognition techniques. *Computers Secur*. 2019;84:225–38.
13. Albakri A, Alabdullah B, Alhayan F. Blockchain-Assisted machine learning with hybrid Metaheuristics-Empowered cyber attack detection and classification model. *Sustainability*. 2023;15(18):22.
14. Roy SD, Debbarma S, Guerrero JM. Machine learning based Multi-Agent system for detecting and neutralizing unseen Cyber-Attacks in AGC and HVDC systems. *IEEE J Emerg Sel Top Circuits Syst*. 2022;12(1):182–93.
15. Hao SQ, Liu D, Baldi S, Yu WW. Unsupervised detection of botnet activities using frequent pattern tree mining. *Complex Intell Syst*. 2022;8(2):761–9.
16. Koduru SS, Machina VSP, Madichetty S. Cyber attacks in Cyber-Physical microgrid systems: A comprehensive review. *Energies*. 2023;16(12):36.
17. Shlomo A, Kalech M, Moskovitch R. Temporal pattern-based malicious activity detection in SCADA systems. *Computers Secur*. 2021;102:17.
18. Xu C, Zhan Y, Wang Z, Yang J. Multimodal fusion based few-shot network intrusion detection system. *Sci Rep*. 2025;15(1):21986.
19. Shi S, Han D, Cui M. A multimodal hybrid parallel network intrusion detection model. *Connection Sci*. 2023;35(1):2227780.
20. Peppes N, Daskalakis E, Alexakis T, Adamopoulou E. A multimodal framework for advanced cybersecurity threat detection using GAN-Driven data synthesis. *Appl Sci*. 2025;15(15):8730.
21. Sikos LF. Cybersecurity knowledge graphs. *Knowl Inf Syst*. 2023;65(9):3511–31.
22. Hu Y, Zou F, Han J, Sun X, Wang Y. Llm-tikg: threat intelligence knowledge graph construction utilizing large Language model. *Computers Secur*. 2024;145:103999.
23. Alharbi H, Hur A, Alkahtani H, Ahmad HF. Enhancing cybersecurity through autonomous knowledge graph construction by integrating heterogeneous data sources. *PeerJ Comput Sci*. 2025;11:e2768.
24. Mitropoulou K, Kokkinos P, Soumplis P, Varvarigos E. Anomaly detection in cloud computing using knowledge graph embedding and machine learning mechanisms. *J Grid Comput*. 2024;22:6.
25. Shin GY, Kim DW, Han MM. Open set recognition with dissimilarity weight for unknown attack detection. *IEEE Access*. 2023;11:102381–90.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.